A Visual Guide to Attention Variants in Modern LLMs

From MHA and GQA to MLA, sparse attention, and hybrid architectures

SEBASTIAN RASCHKA, PHD
MAR 22, 2026

I had originally planned to write about DeepSeek V4. Since it still hasn't been released, the time to work on something that had been on my list for a while, namely, collecting, organizing, and refining the different LLM architectures I have covered over the past fe years.

So, over the last two weeks, I turned that effort into an LLM architecture gallery (with 4 entries at the time of this writing), which combines material from earlier articles with se important architectures I had not documented yet. Each entry comes with a visual mod card, and I plan to keep the gallery updated regularly.

You can find the gallery here: https://sebastianraschka.com/llm-architecture-gallery/

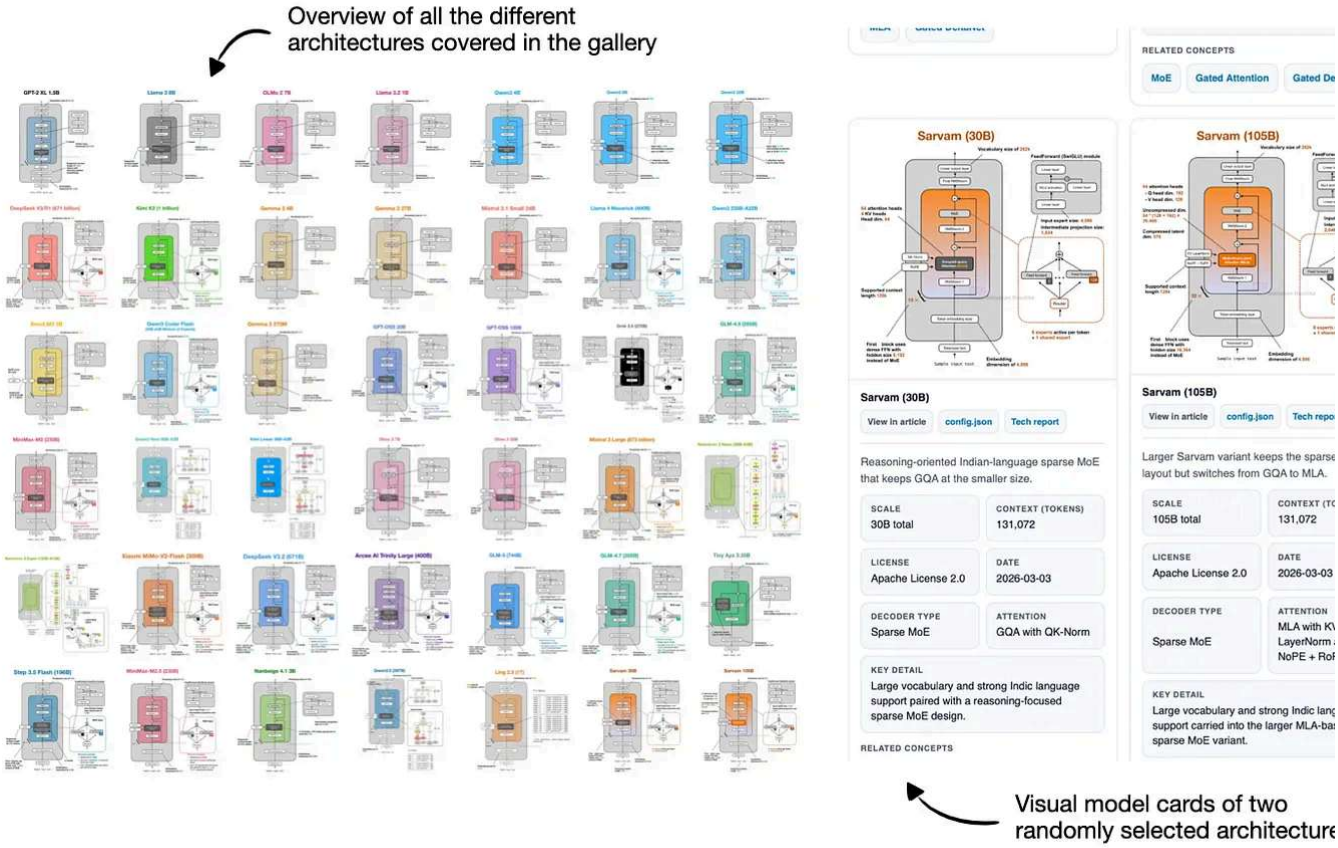


Figure 1: Overview of the LLM architecture gallery and its visual model cards.

After I shared the initial version, a few readers also asked whether there would be a pos version. So, there is now a poster version via Redbubble. I ordered the Medium size (26 23.4 in) to check how it looks in print, and the result is sharp and clear. That said, some smallest text elements are already quite small at that size, so I would not recommend th smaller versions if you intend to have everything readable.

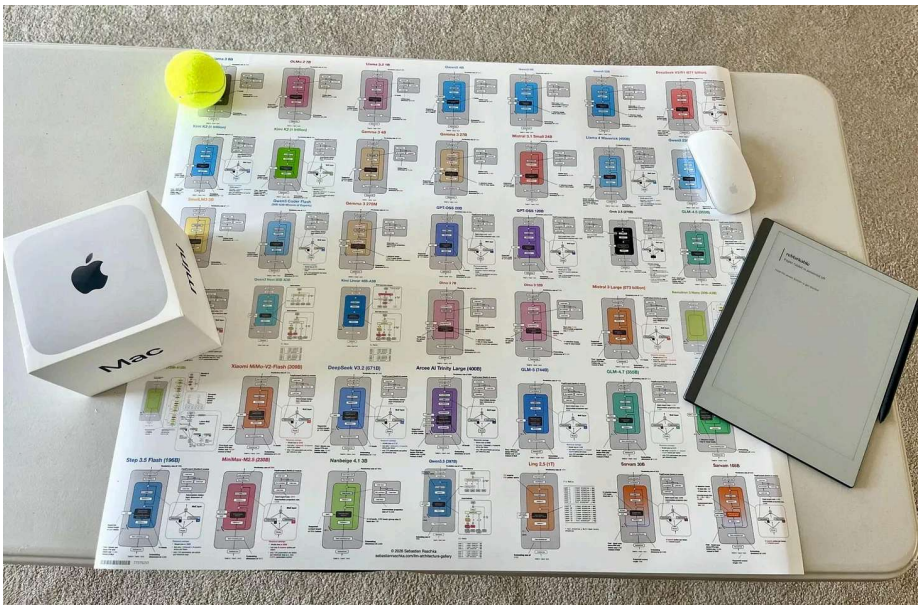


Figure 2: Poster version of the architecture gallery with some random objects for scale.

Alongside the gallery, I was/am also working on short explainers for a few core LLM concepts.

So, in this article, I thought it would be interesting to recap all the recent attention variar that have been developed and used in prominent open-weight architectures in recent

My goal is to make the collection useful both as a reference and as a lightweight learni resource. I hope you find it useful and educational!

1. Multi-Head Attention (MHA)

Self-attention lets each token look at the other visible tokens in the sequence, assign tl weights, and use those weights to build a new context-aware representation of the inp

Multi-head attention (MHA) is the standard transformer version of that idea. It runs sev self-attention heads in parallel with different learned projections, then combines their c into one richer representation.

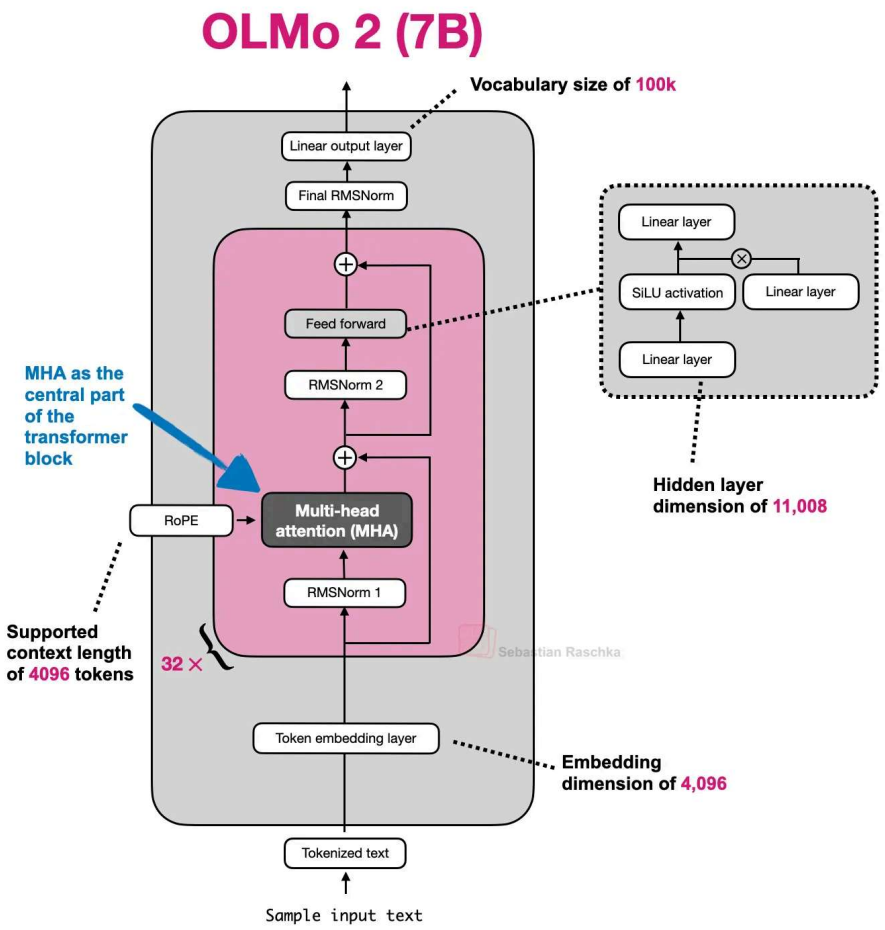


Figure 3: Olmo 2 as an example architecture using MHA.

The sections below start with a whirlwind tour of explaining self-attention to explain M more meant as a quick overview to set the stage for related attention concepts like gro query attention, sliding window attention, and so on. If you are interested in a longer, mo detailed self-attention coverage, you might like my longer Understanding and Coding S Attention, Multi-Head Attention, Causal-Attention, and Cross-Attention in LLMs article

EXAMPLE ARCHITECTURES

GPT-2, OLMo 2 7B, and OLMo 3 7B

1.2 Historical Tidbits And Why Attention Was Invented

Attention predates transformers and MHA. Its immediate background is encoder-deco RNNs for translation.

In those older systems, an encoder RNN would read the source sentence token by toke compress it into a sequence of hidden states, or in the simplest version into one final st Then the decoder RNN had to generate the target sentence from that limited summary worked for short and simple cases, but it created an obvious bottleneck once the relev information for the next output word lived somewhere else in the input sentence.

In short, the limitation is that the hidden state can't store infinitely much information or context, and sometimes it would be useful to just refer back to the full input sequence.

The translation example below shows one of the limitations of this idea. For instance, a sentence can preserve many locally reasonable word choices and still fail as a translati when the model treats the problem too much like a word-by-word mapping. (The top p shows an exaggerated example where we translate the sentence word by word; obviou the grammar in the resulting sentence is wrong.) In reality, the correct next word depen sentence-level structure and on which earlier source words matter at that step. Of cou this could still be translated fine with an RNN, but it would struggle with longer sequen knowledge retrieval tasks because the hidden state can only store so much informatio mentioned earlier.

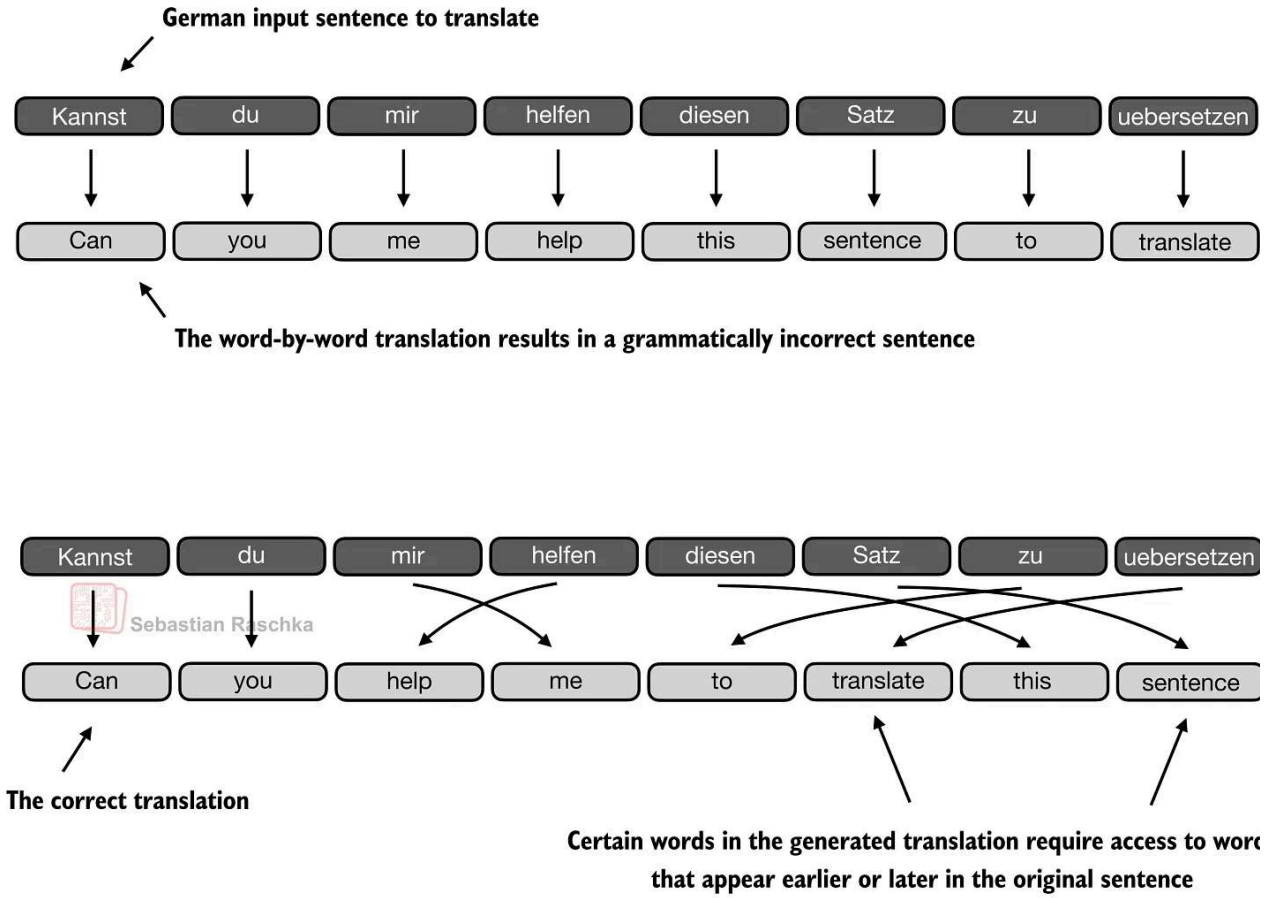


Figure 4: Translation can fail even when many individual word choices look reasonable because sentence-level structure still matters (Original source *LLMs-from-scratch*).

The next figure shows that change more directly. When the decoder is producing an ou token, it should not be limited to one compressed memory path. It should be able to rea back to the more relevant input tokens directly.

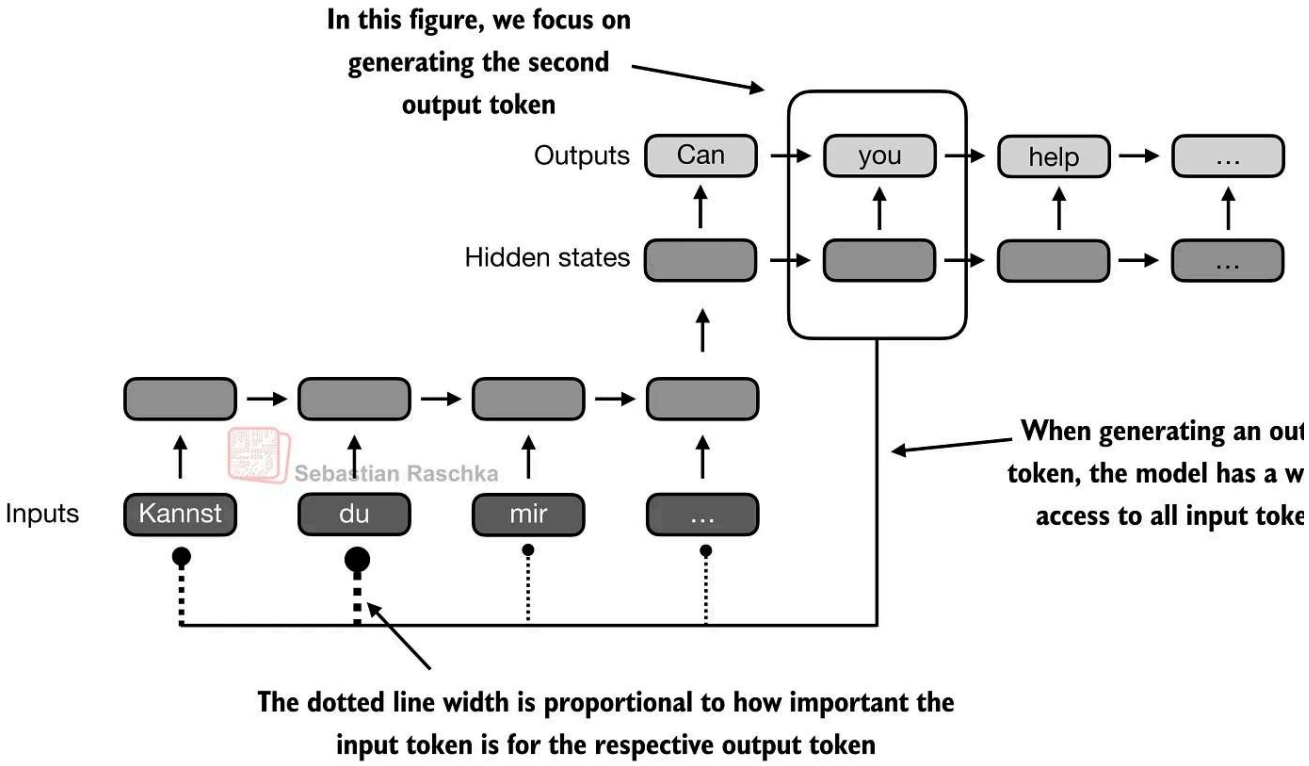


Figure 5: Attention breaks the RNN bottleneck by letting the current output position revisit the full input sequence instead of relying on one compressed state alone (Original source *LLMs-from-scratch*).

Transformers keep that core idea from the aforementioned attention-modified RNN bu remove the recurrence. In the classic *Attention Is All You Need* paper, attention becom main sequence-processing mechanism itself (instead of being just part of an RNN enc decoder.)

In transformers, that mechanism is called self-attention, where each token in the seque computes weights over all other tokens and uses them to mix information from those t into a new representation. Multi-head attention is the same mechanism run several tim parallel.

1.3 The Masked Attention Matrix

For a sequence of T tokens, attention needs one row of weights per token, so overall we $T \times T$ matrix.

Each row answers a simple question. When updating this token, how much should eac visible token matter? In a decoder-only LLM, future positions are masked out, which is

the upper-right part of the matrix is grayed out in the figure below.

Self-attention is fundamentally about learning these token-to-token weight patterns, u causal mask, and then using them to build context-aware token representations.

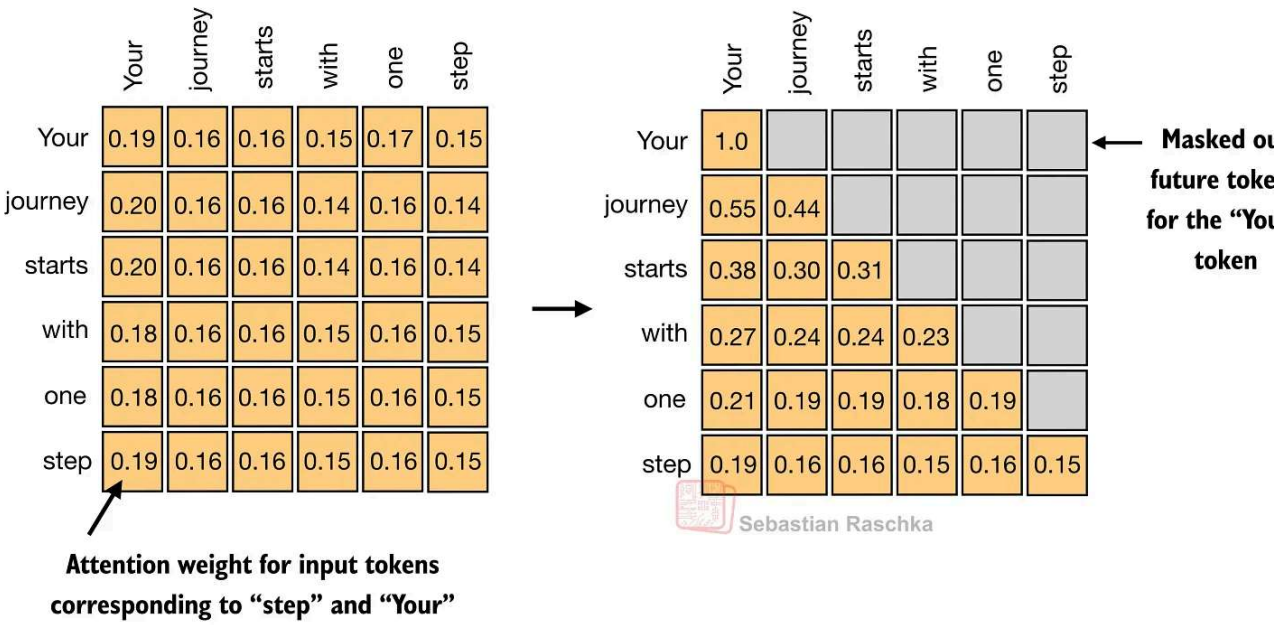


Figure 6: A concrete masked attention matrix where each row belongs to one token, each entry is an attention weight, and future-token entries are removed by the causal mask (Original source *Understanding and Coding Self-Attention*).

1.4 Self-Attention Internals

The next figure shows how the transformer computes the attention matrix (A) from the embeddings X, which is then used to produce the transformed inputs (Z).

Here Q, K, and V stand for queries, keys, and values. The query for a token represents wh that token is looking for, the key represents what each token makes available for match and the value represents the information that gets mixed into the output once the atter weights have been computed.

The steps are as follows:

- W_q, W_k, and W_v are weight matrices that project the input embeddings into Q, K, and
- QK^T produces the raw token-to-token relevance scores

- softmax converts those scores into the normalized attention matrix A that we disc in the previous section
- A is applied to V to produce the output matrix Z

Note that the attention matrix is not a separate hand-written object. It emerges from Q, softmax.

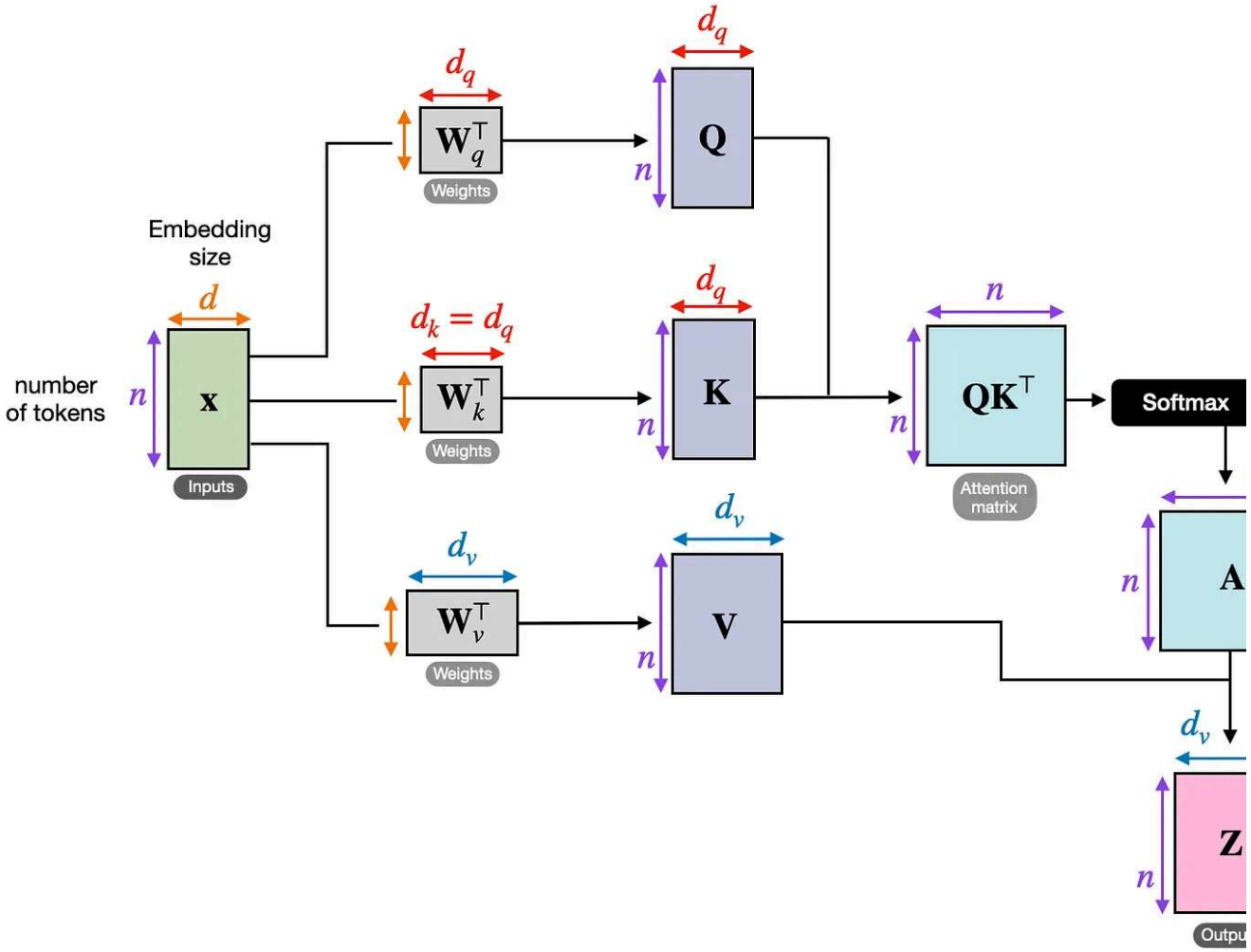


Figure 7: The full single-head pipeline, from input embeddings X to the normalized attention matrix A and output representations Z (Original source *Understanding and Coding Self-Attention*).

The next figure shows the same concept as the previous figure but the attention matri computation is hidden inside the "scaled-dot-product attention" box, and we perform t computation only for one input token instead of all input tokens. This is to show a comp form of self-attention with a single head before extending this to multi-head attention i next section.

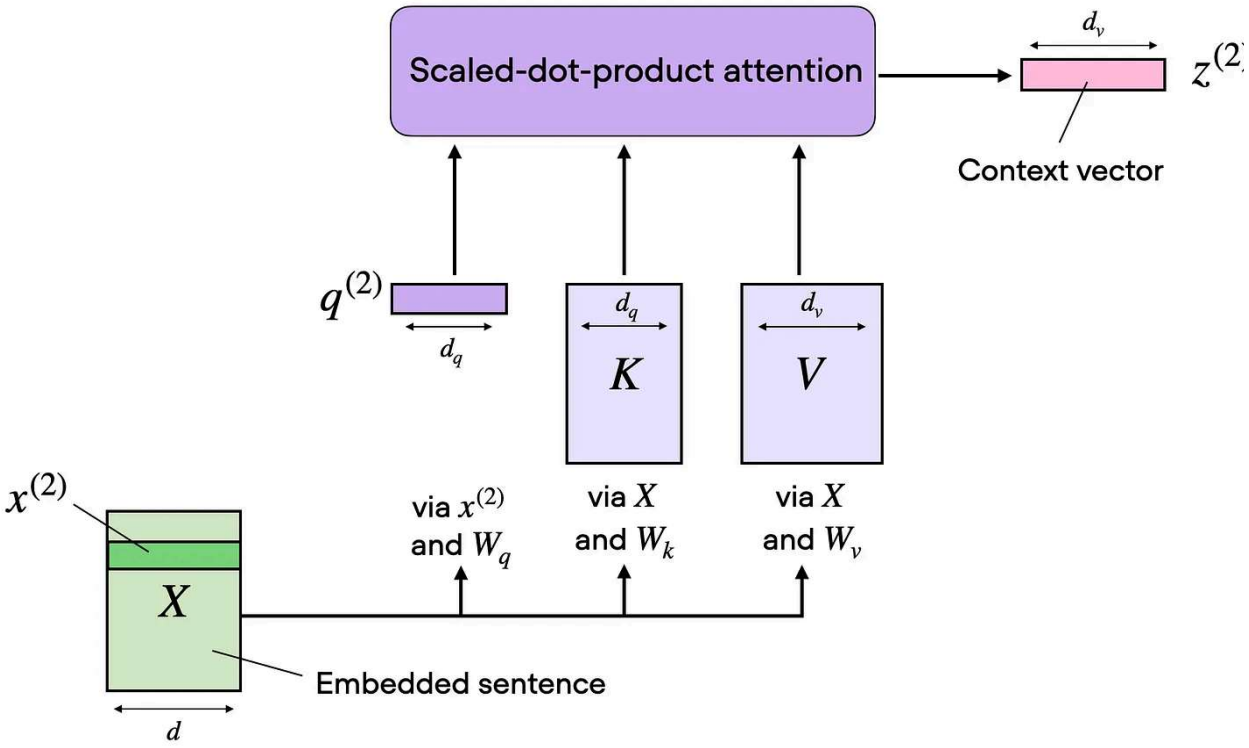


Figure 8: One attention head is already a complete mechanism. One set of learned projections produces one attention matrix and one context-aware output stream (Original source *Understanding and Coding Self-Attention*).

1.5 From One Head To Multi-Head Attention

One set of Wq/Wk/Wv matrices gives us one attention head, which means one attention and one output matrix Z. (This concept was illustrated in the previous section.)

Multi-head attention simply runs several of these heads in parallel with different learne projection matrices.

This is useful because different heads can specialize in different token relationships. O head might focus on short local dependencies, another on broader semantic links, and another on positional or syntactic structure.

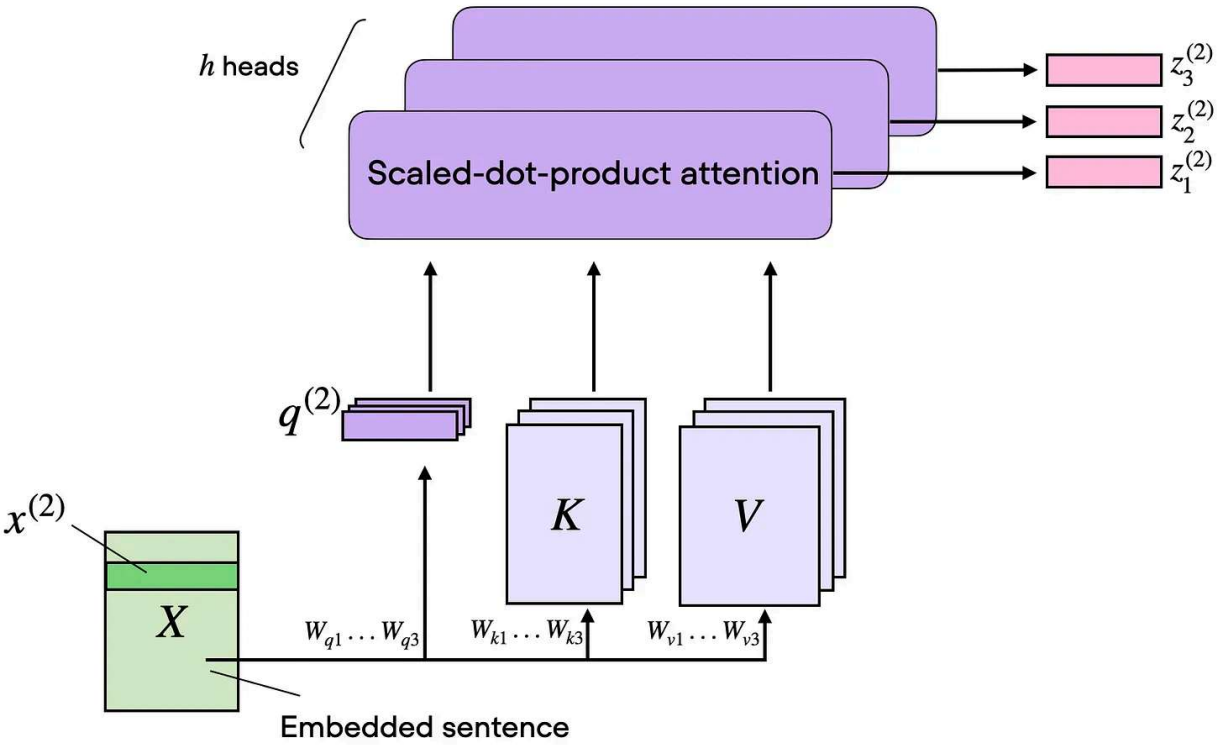


Figure 9: Multi-head attention keeps the same basic attention recipe, but repeats it across several heads in parallel so the model can learn several token-to-token patterns at once (Original source *Understanding and Coding Self-Attention*).

2. Grouped-Query Attention (GQA)

Grouped-query attention is an attention variant derived from standard MHA. It was introduced in the 2023 paper *GQA: Training Generalized Multi-Query Transformer M from Multi-Head Checkpoints* by Joshua Ainslie and colleagues.

Instead of giving every query head its own keys and values, it lets several query heads the same key-value projections, which makes KV caching much cheaper (primarily as memory reduction) without changing the overall decoder recipe very much.

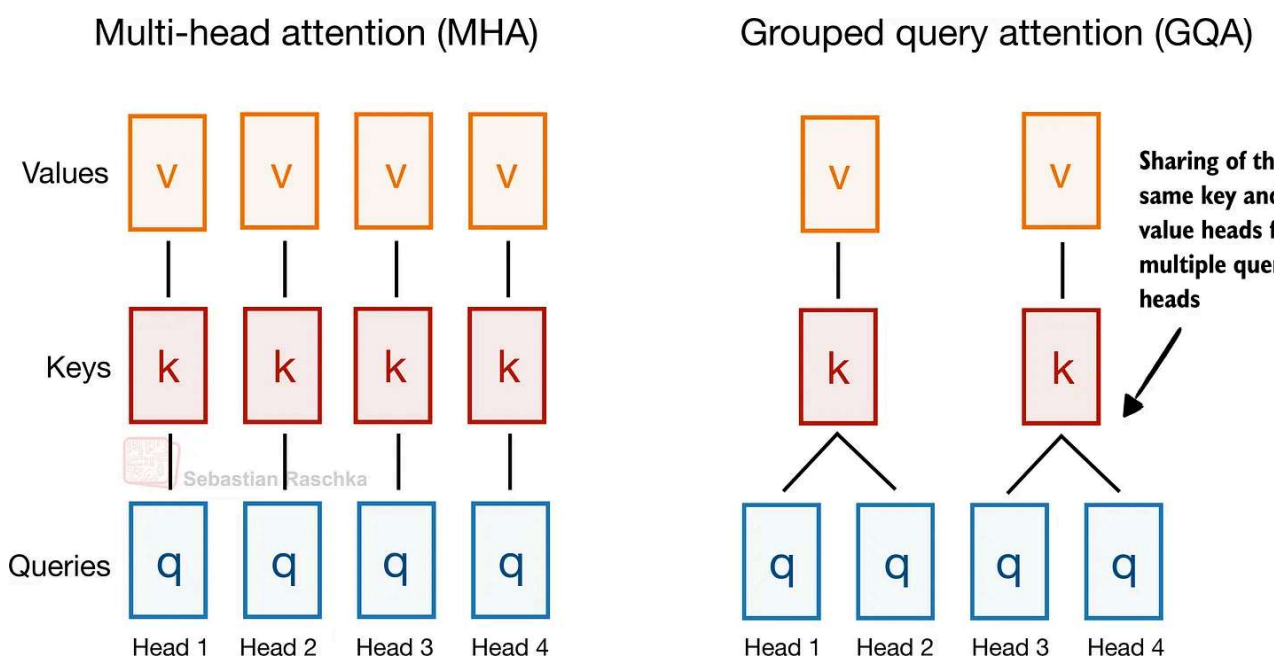


Figure 10: GQA keeps the same overall attention pattern as MHA, but collapses the number of key-value heads by sharing them across multiple query heads (Original source: *The Big LLM Architecture Comparison*).

EXAMPLE ARCHITECTURES

Dense: Llama 3 8B, Qwen3 4B, Gemma 3 27B, Mistral Small 3.1 24B, SmolLM3 3B, an Aya 3.35B.
Sparse (Mixture-of-Experts): Llama 4 Maverick, Qwen3 235B-A22B, Step 3.5 Flash 1 and Sarvam 30B.

2.1 Why GQA Became Popular

In my architecture comparison article, I framed GQA as the new standard replacement classic multi-head attention (MHA). The reason is that standard MHA gives every hea own keys and values, which is more optimal from a modeling perspective but expensiv we have to keep all of that state in the KV cache during inference.

In GQA, we keep a larger set of query heads, but we reduce the number of key-value h and let multiple queries share them. That lowers both parameter count and KV-cache t without making drastic implementation changes like multi-head latent attention (MLA) which will be discussed later.

In practice, that made and keeps it a very popular choice for labs that wanted somethir cheaper than MHA but simpler to implement than newer compression-heavy alternati like MLA.

2.2 GQA Memory Savings

GQA results in big savings in KV storage, since the fewer key-value heads we keep per the less cached state we need per token. That is why GQA becomes more useful as sequence length grows.

GQA is also a spectrum. If we reduce all the way down to one shared K/V group, we are effectively in multi-query attention territory, which is even cheaper but can hurt modeli quality more noticeably. The sweet spot is usually somewhere in between multi-query attention (1 shared group) and MHA (where K/V groups are equal to the number of que where the cache savings are large but the modeling degradation relative to MHA stays modest.

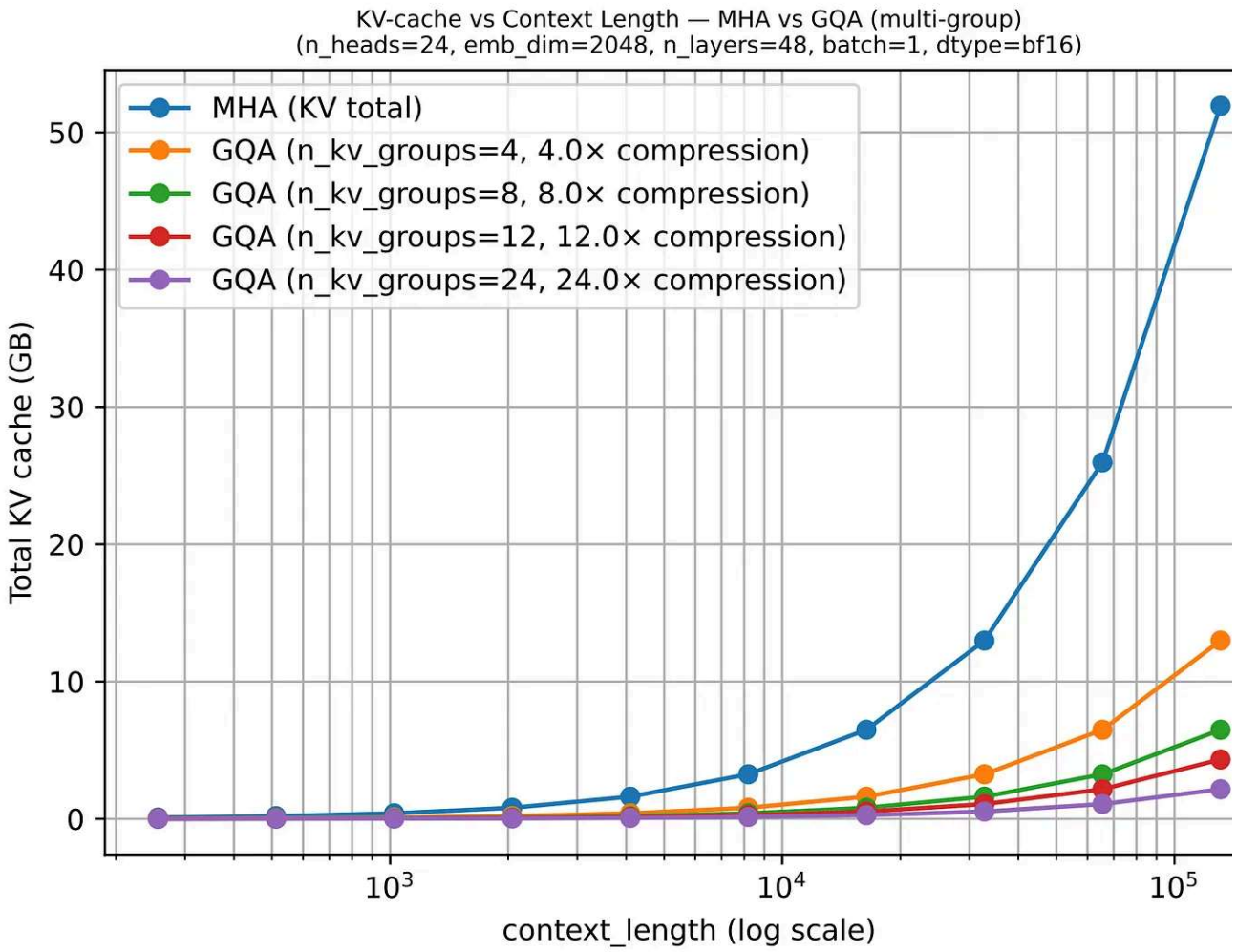


Figure 11: Lower is better. Once the context window grows, KV-cache savings become more pronounced. (Original source: *LLMs-from-scratch* GQA materials)

2.3 Why GQA Still Matters In 2026

More advanced variants such as MLA are becoming popular because they can offer be modeling performance at the same KV efficiency levels (e.g., as discussed in the ablati studies of the DeepSeek-V2 paper), but they also involve a more complicated implementation and a more complicated attention stack.

GQA remains appealing because it is robust, easier to implement, and also easier to tra (since there are fewer hyperparameter tunings necessary, based on my experience).

That is why some of the newer releases still stay deliberately classic here. E.g., in my Sp Architectures article, I mentioned that MiniMax M2.5 and Nanbeige 4.1 as models that remained very classic, using only grouped-query attention without piling on other effic

tricks. Sarvam is a particularly useful comparison point as well: the 30B model keeps c GQA, while the 105B version switches to MLA.

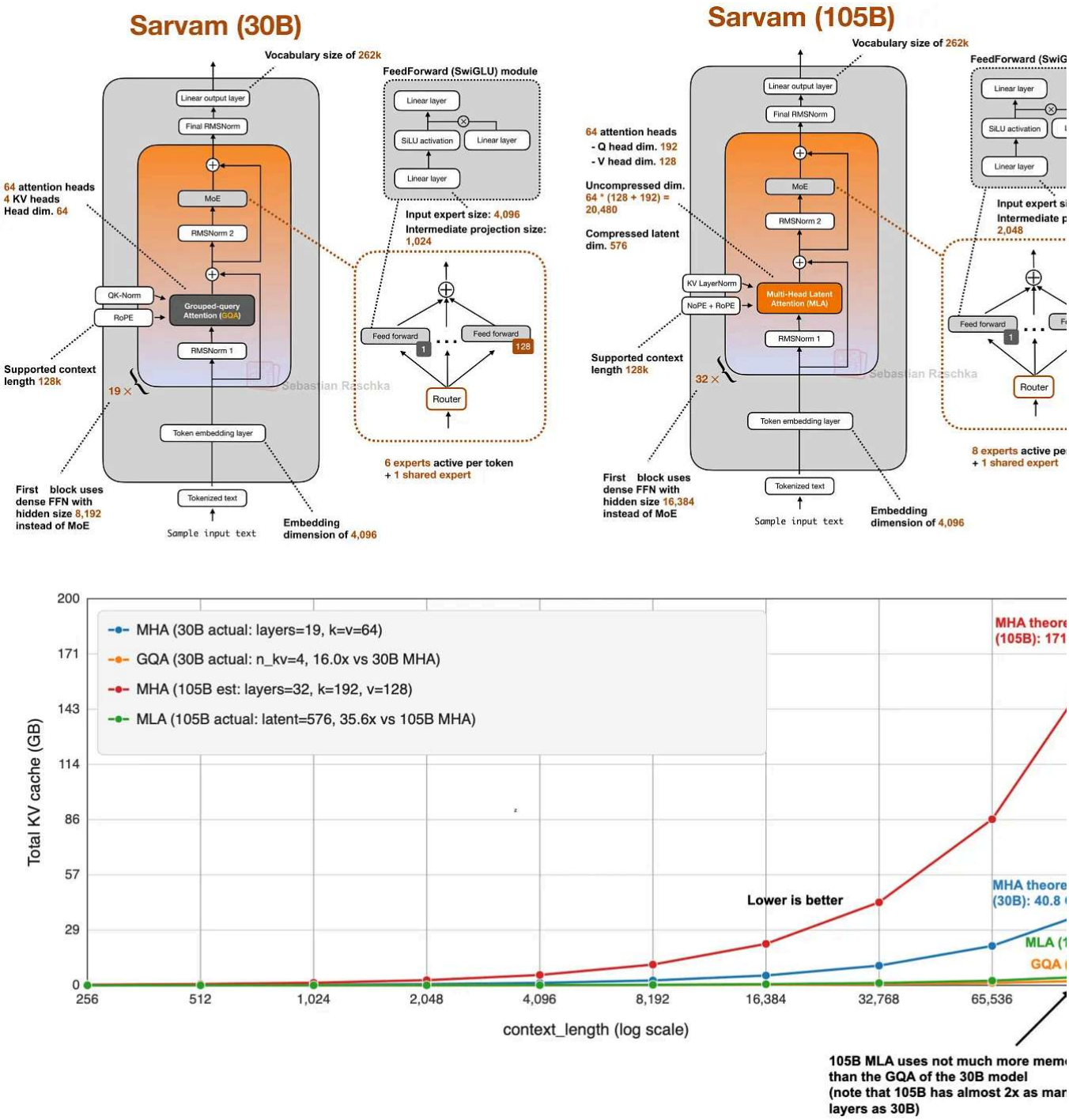


Figure 12: Total KV cache sizes for 105B Sarvam (using MLA) versus 30B Sarvam (using GQA), versus using plain MHA.

3. Multi-Head Latent Attention (MLA)

The motivation behind Multi-head Latent Attention (MLA) is similar to Grouped-Query Attention (GQA). Both are solutions for reducing KV-cache memory requirements. The difference between GQA and MLA is that MLA shrinks the cache by compressing wha stored rather than by reducing how many K/Vs are stored by sharing heads.

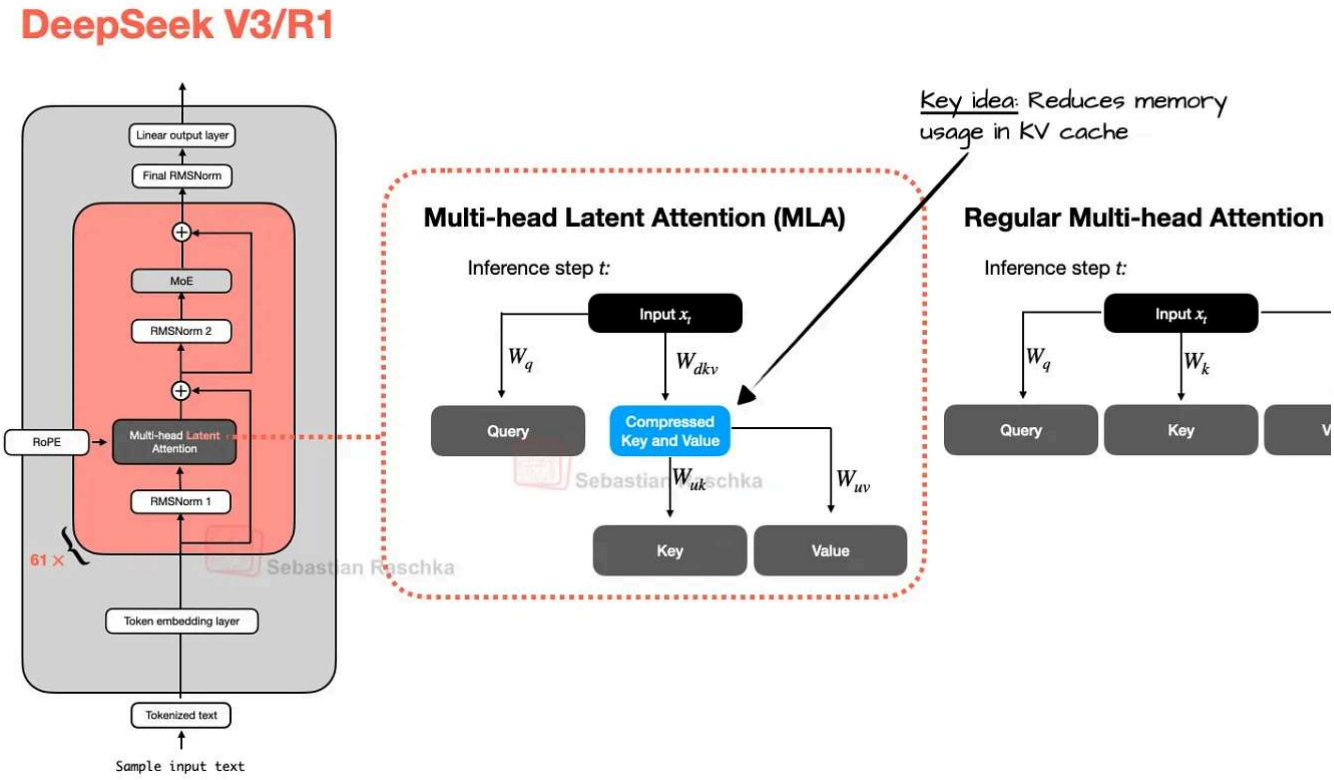


Figure 13: Unlike GQA, MLA does not reduce KV cost by grouping heads. It reduces it by caching a compressed latent representation. Note that it is also applied to the query, which is not shown for simplicity (Original source: *The Big LLM Architecture Comparison*).

MLA, originally proposed in the DeepSeek-V2 paper, became such a defining DeepSe idea (especially after DeepSeek-V3 and R1). It is more complicated to implement than more complicated to serve, but nowadays also often more compelling once model size context length get large enough that cache traffic starts to dominate, because at the sa rate of memory reduction, it could maintain better modeling performance (more on tha

EXAMPLE ARCHITECTURES

DeepSeek V3, Kimi K2, GLM-5, Ling 2.5, Mistral Large 3, and Sarvam 105B

3.1 Compression, Not Sharing

Instead of caching full-resolution key and value tensors as in MHA and GQA, MLA stor latent representation and reconstructs the usable state when needed. Essentially, it is a cache compression strategy embedded inside attention, as illustrated in the previous f

The figure below shows the savings compared to regular MHA.

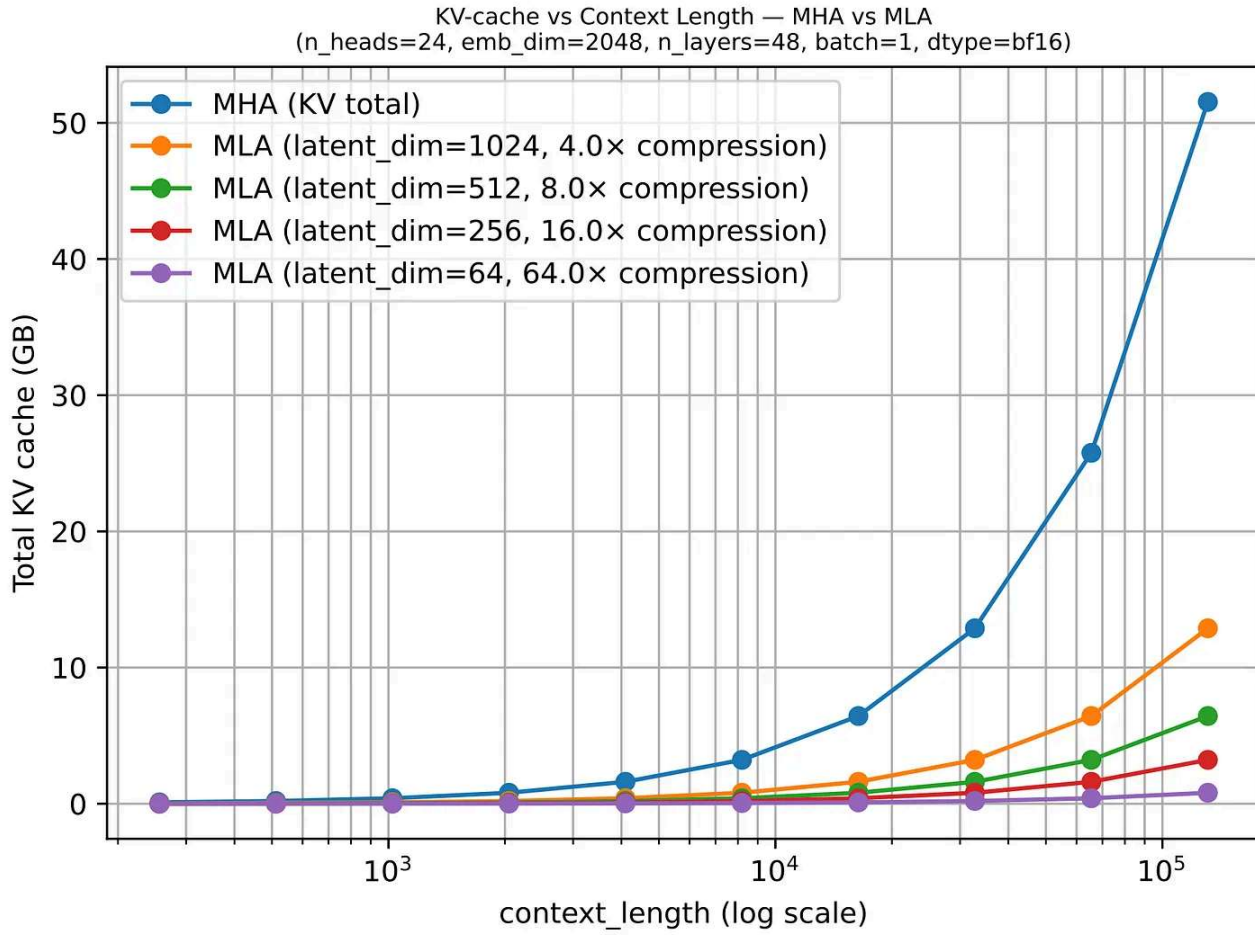
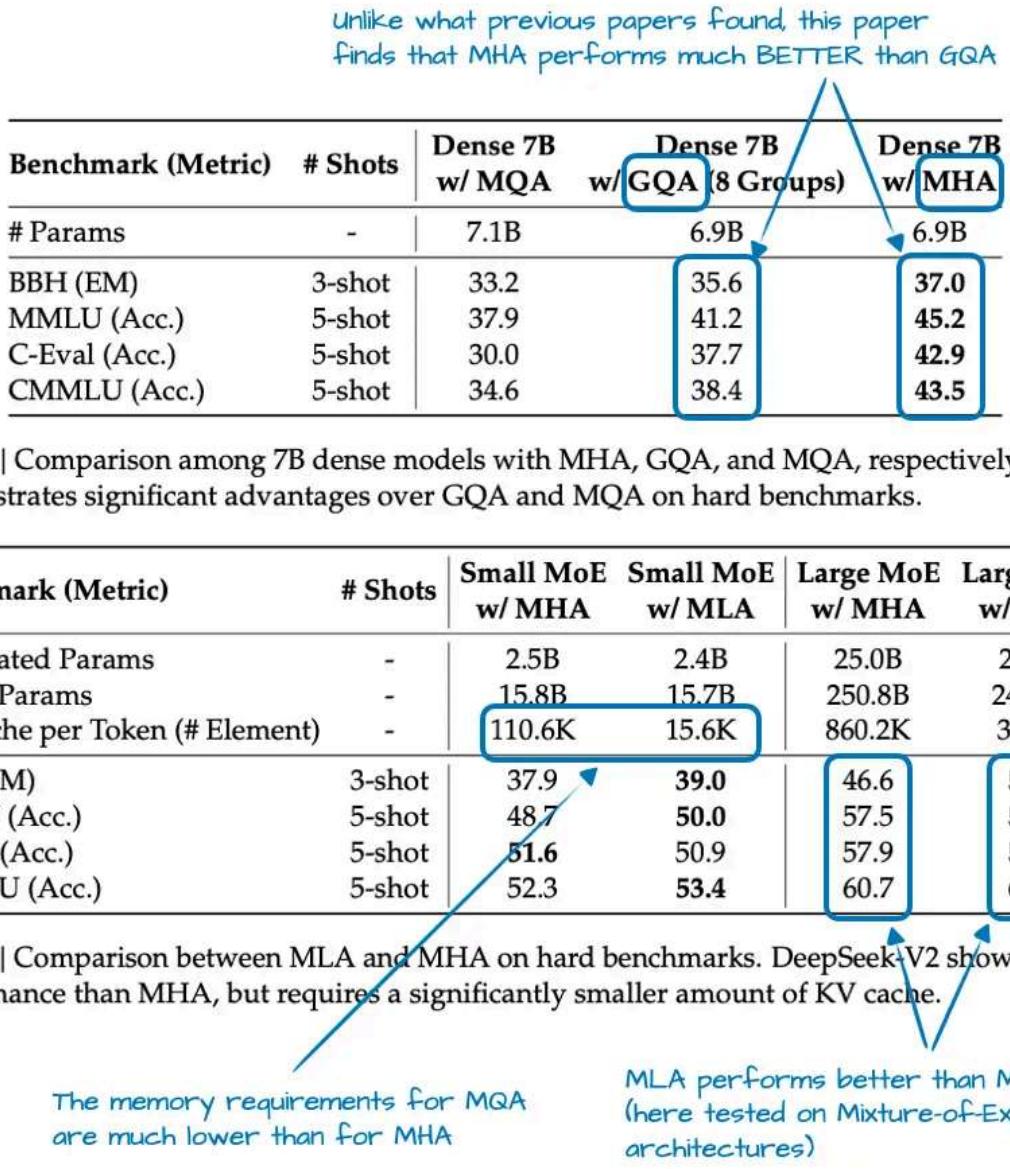


Figure 14: Once context length grows, the savings from caching a latent representation instead of full K/V tensors become very visible (Original source: *LLMs-from-scratch* MLA section).

3.2 MLA Ablation Studies

The DeepSeek-V2 paper provided some ablations where GQA looked worse than MH terms of modeling performance, while MLA held up much better and could even outpe MHA when tuned carefully. That is a much stronger justification than "it (also) saves memory."

In other words, MLA is a preferable attention mechanism for DeepSeek not just becaus was efficient, but because it looked like a quality-preserving efficiency move at large s (But colleagues also told me that MLA only works well at a certain size. For smaller mo let's say <100B, GQA seems to work better, or, is at least easier to tune and get right.)



Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	**37.0**
MMLU (Acc.)	5-shot	37.9	41.2	**45.2**
C-Eval (Acc.)	5-shot	30.0	37.7	**42.9**
CMMLU (Acc.)	5-shot	34.6	38.4	**43.5**

Table 8 | Comparison among 7B dense models with MHA, GQA, and MQA, respectively. MHA demonstrates significant advantages over GQA and MQA on hard benchmarks.

Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	**39.0**	46.6	**50.7**
MMLU (Acc.)	5-shot	48.7	**50.0**	57.5	**59.0**
C-Eval (Acc.)	5-shot	**51.6**	50.9	57.9	**59.2**
CMMLU (Acc.)	5-shot	52.3	**53.4**	60.7	**62.5**

Table 9 | Comparison between MLA and MHA on hard benchmarks. DeepSeek-V2 shows better performance than MHA, but requires a significantly smaller amount of KV cache.

Figure 15: GQA drops below MHA here, while MLA remains competitive and can even slightly outperform it. Underlying paper: DeepSeek-V2.

Below is again the comparison between GQA in 30B Sarvam versus MLA in 105B Sar

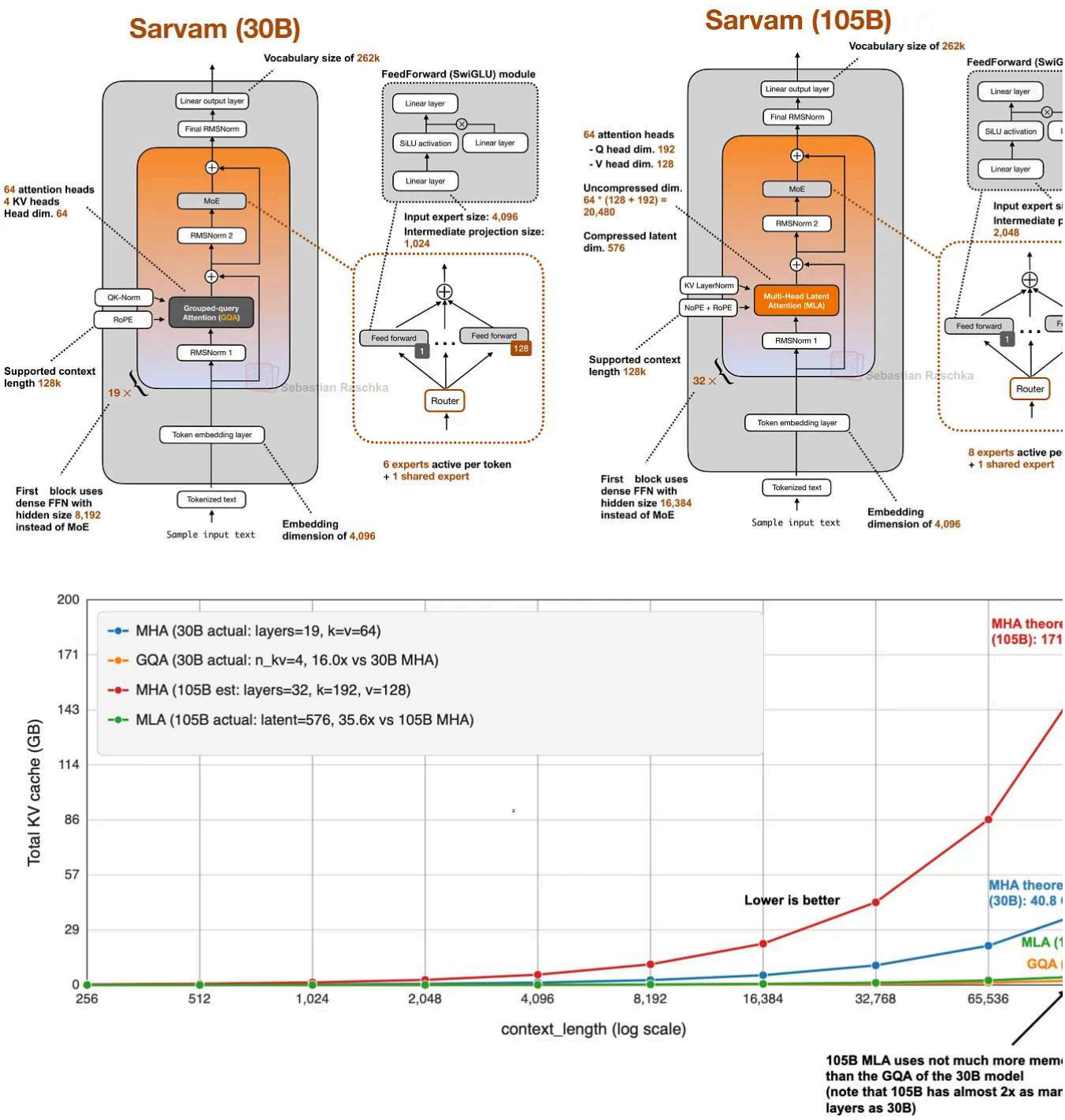


Figure 16: GQA and MLA are solving the same bottleneck from different directions. The tradeoff is simplicity versus better modeling performance for larger models.

3.3 How MLA Spread After DeepSeek

Once DeepSeek V3/R1, V3.1 etc. normalized the design after its introduction in V2, it st showing up in a second wave of architectures. Kimi K2 kept the DeepSeek recipe and s it up. GLM-5 adopted MLA together with DeepSeek Sparse Attention (from DeepSeek Ling 2.5 paired MLA with a linear-attention hybrid. Sarvam released two models where 30B model stayed with classic GQA and the 105B model switched to MLA.

That last pair is particularly useful as it puts the technical-complexity discussion aside the Sarvam team implemented both variants and deliberately chose to then use GQA f variant and MLA for the other. So, in a sense, that makes MLA feel less like a theoretica alternative and more like a concrete architectural upgrade path once a family scales up

4. Sliding Window Attention (SWA)

Sliding window attention reduces the memory and compute cost of long-context infer by limiting how many previous tokens each position can attend to. Instead of attending entire prefix, each token only attends to a fixed window of recent tokens around its pos Because attention is restricted to a local token neighborhood, this mechanism is often referred to as local attention.

Some architectures combine these local layers with occasional global attention layers information can still propagate across the entire sequence.

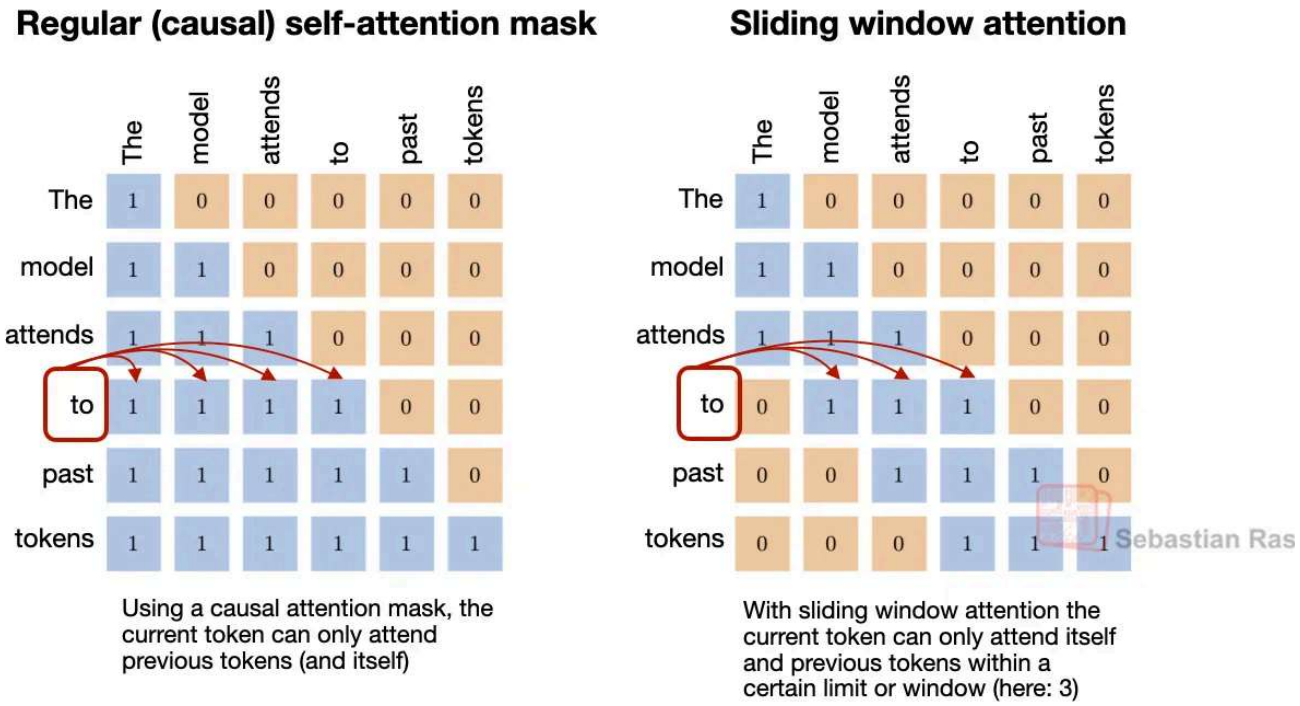


Figure 17: The conceptual shift is simple. Regular attention is global attention, while sliding-window attention is local attention. Global attention lets every token see the full prefix; SWA turns many of those layers into local attention layers (Original source: *The Big LLM Architecture Comparison*).

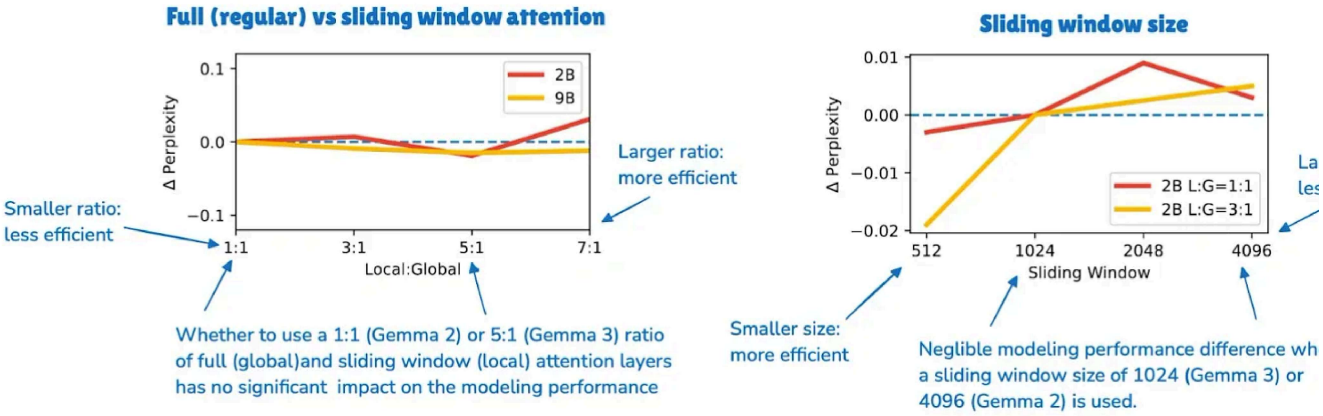
EXAMPLE ARCHITECTURES

Gemma 3 27B, OLMo 3 32B, Xiaomi MiMo-V2-Flash, Arcee Trinity, Step 3.5 Flash, and Aya

4.1 Gemma 3 As A Reference Point

Gemma 3 is still one of the clearest recent SWA examples because it is easy to compa against Gemma 2. Gemma 2 already used a hybrid attention setup with a 1:1 ratio betw local and global layers and a 4096-token window. Gemma 3 pushed this further to a 5 and reduced the window size to 1024.

The key finding was not that local attention is cheaper, because that was already know Here, the more interesting takeaway from the Gemma 3 ablation study was that using t more aggressively seemed to hurt modeling performance only slightly.



The Gemma ablation study suggests that the smaller window and more aggressive local:global ratio have little effect on perplexity. Underlying paper: Gemma 3 article (Original source: *The Big LLM Architecture Comparison*).

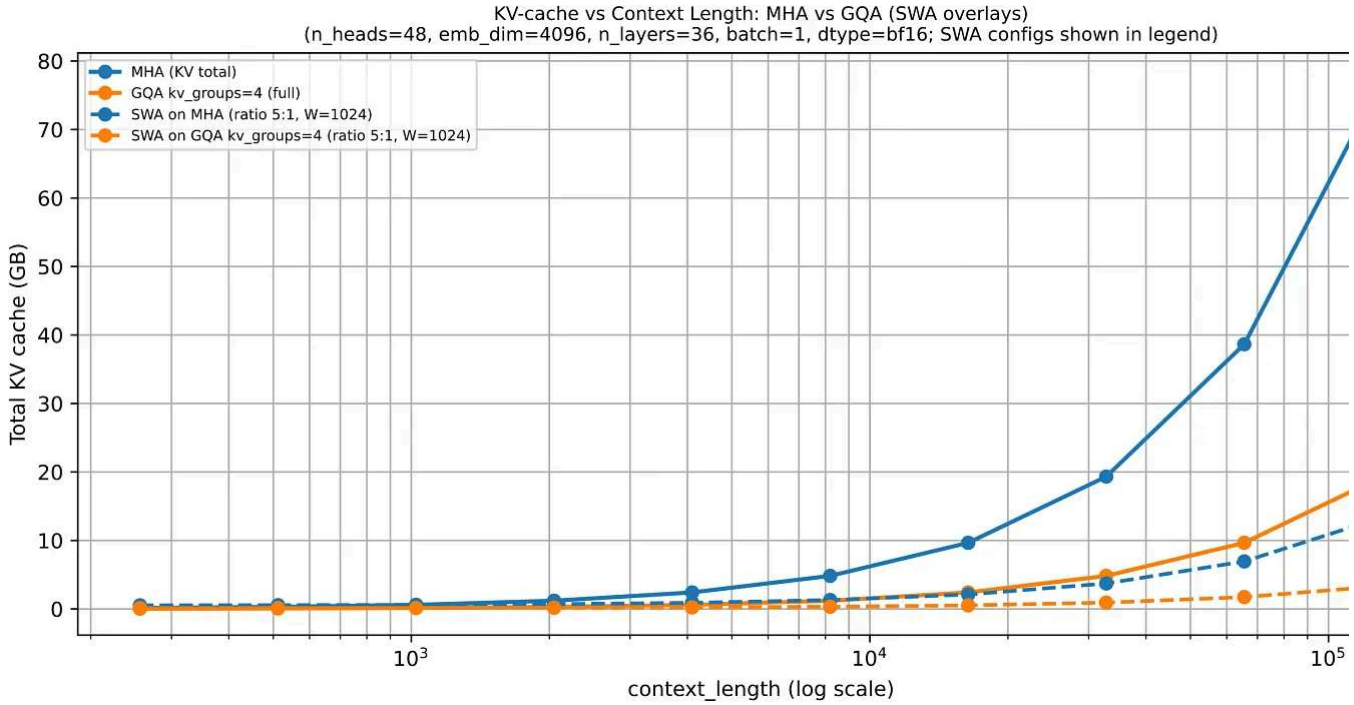
4.2 The Ratio And Window Size

In practice, saying that a model "uses SWA" does not mean it relies on SWA alone. Wha usually matters are the local-to-global layer pattern and the attention window size. For example:

- Gemma 3 and Xiaomi use a 5:1 local-to-global pattern.

- OLMo 3 and Arcee Trinity use a 3:1 pattern.
- Xiaomi also uses a window size of 128, which is much smaller, and therefore more aggressive, than Gemma's 1024.

SWA is essentially a knob that can be tuned more or less aggressively.



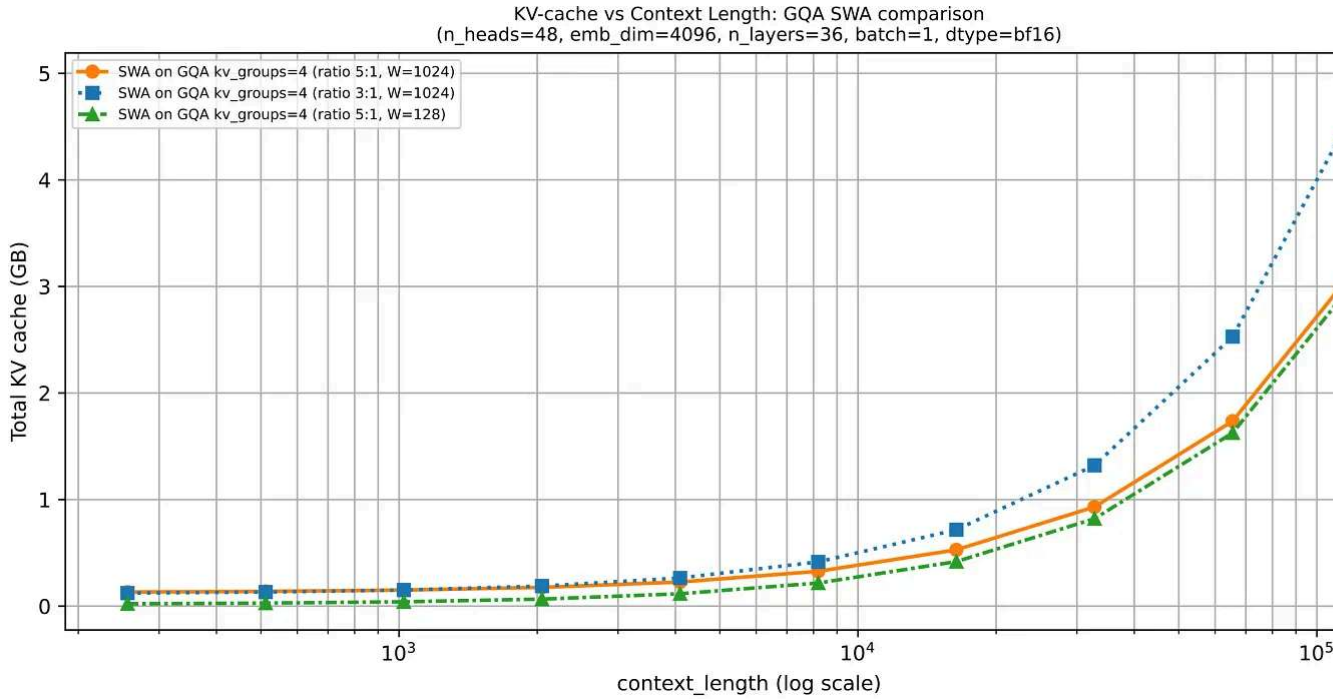


Figure 18: The long-context savings come from turning many full-attention layers into local ones, which reduces how much cached context those layers need to consider (Original source: *LLMs-from-scratch* SWA materials).

4.3 Combining SWA with GQA

SWA often appears together with GQA because the two ideas address different parts of same inference problem. SWA reduces how much context a local layer has to consider reduces how much key-value state each token contributes to the cache.

That is why many recent dense models use both rather than treating them as alternati Gemma 3 is again a good reference point here, since it combines sliding window attent with grouped-query attention in the same architecture.

5. DeepSeek Sparse Attention (DSA)

DeepSeek Sparse Attention is one of the architectural changes that appeared in the DeepSeek V3.2 line and later showed up again in GLM-5.

Specifically, DeepSeek V3.2 combines it with Multi-head Latent Attention (MLA), and 5 adopts the same pair for the same general reason, namely, reducing inference cost w context lengths get large.

EXAMPLE ARCHITECTURES

DeepSeek V3.2 and GLM-5

5.1 Changes Relative To Sliding-Window Attentic

In sliding-window attention, the current token does not attend to the full prefix but only fixed local window. This is the same broad idea behind DeepSeek Sparse Attention, wh each token also only attends to a subset of previous tokens.

However, the selected tokens are not determined by a fixed-width local window. Instea DeepSeek Sparse Attention uses a learned sparse pattern. In short, it uses an indexer-selector setup, where a lightning indexer computes relevance scores, and a token selec keeps only a smaller set of high-scoring past positions.

The way the subset of tokens is selected is the main difference from sliding-window attention. Sliding-window attention hard-codes locality. DeepSeek Sparse Attention st limits attention to a subset, but it lets the model decide which prior tokens are worth revisiting.

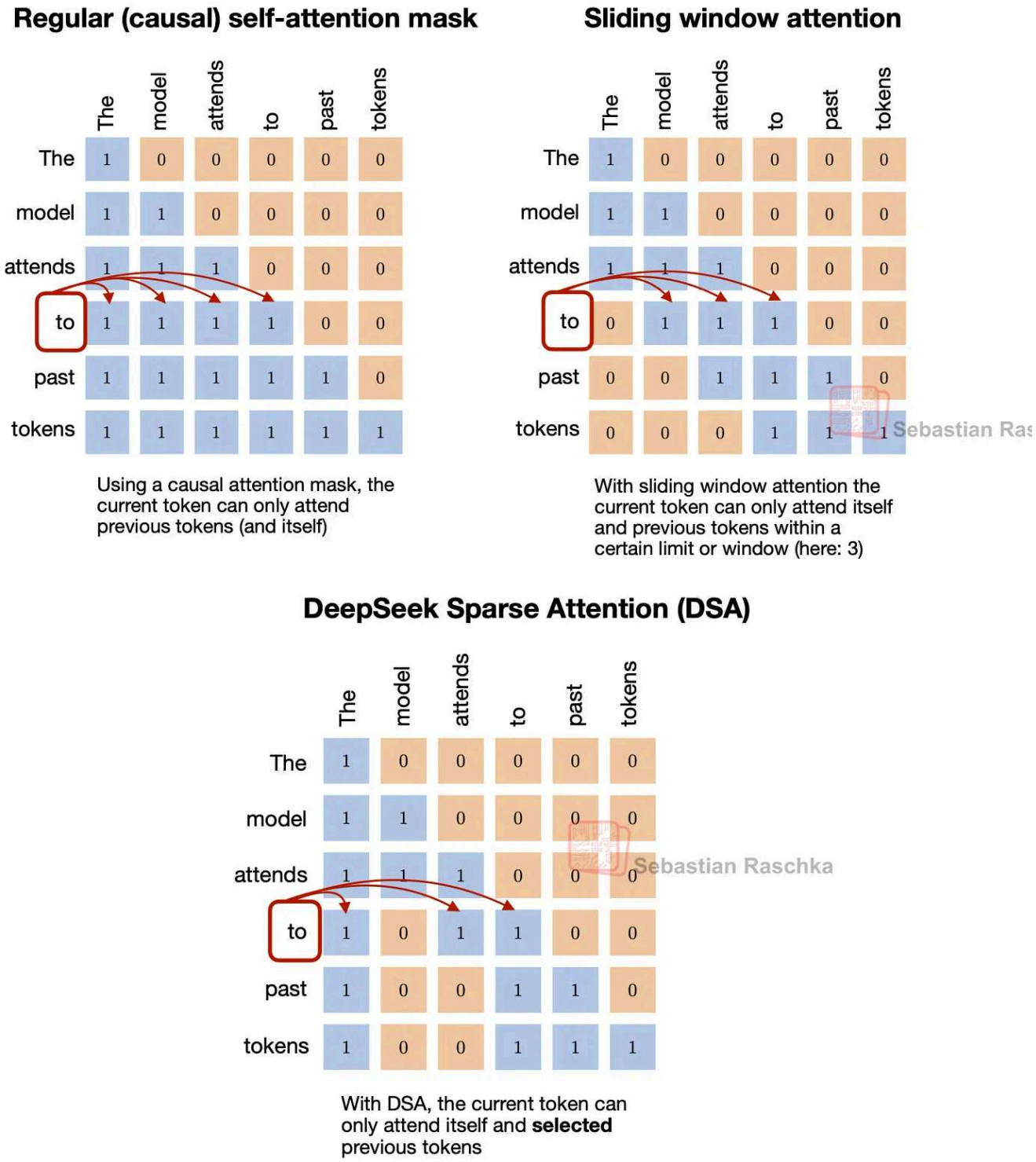


Figure 19: Similar to sliding-window attention, DeepSeek Sparse Attention also restricts each token to a subset of prior tokens, but does not do so with a fixed local window (Original source: *From DeepSeek V3 to V3.2: Architecture, Sparse Attention, and RL Updates*).

5.2 DeepSeek Sparse Attention and MLA

DeepSeek V3.2 uses both Multi-head Latent Attention (MLA) and DeepSeek Sparse Attention. MLA reduces KV-cache cost by compressing what gets stored. DeepSeek S Attention reduces how much of the prior context the model has to revisit. Put differentl optimizes the cache representation, the other optimizes the attention pattern on top of

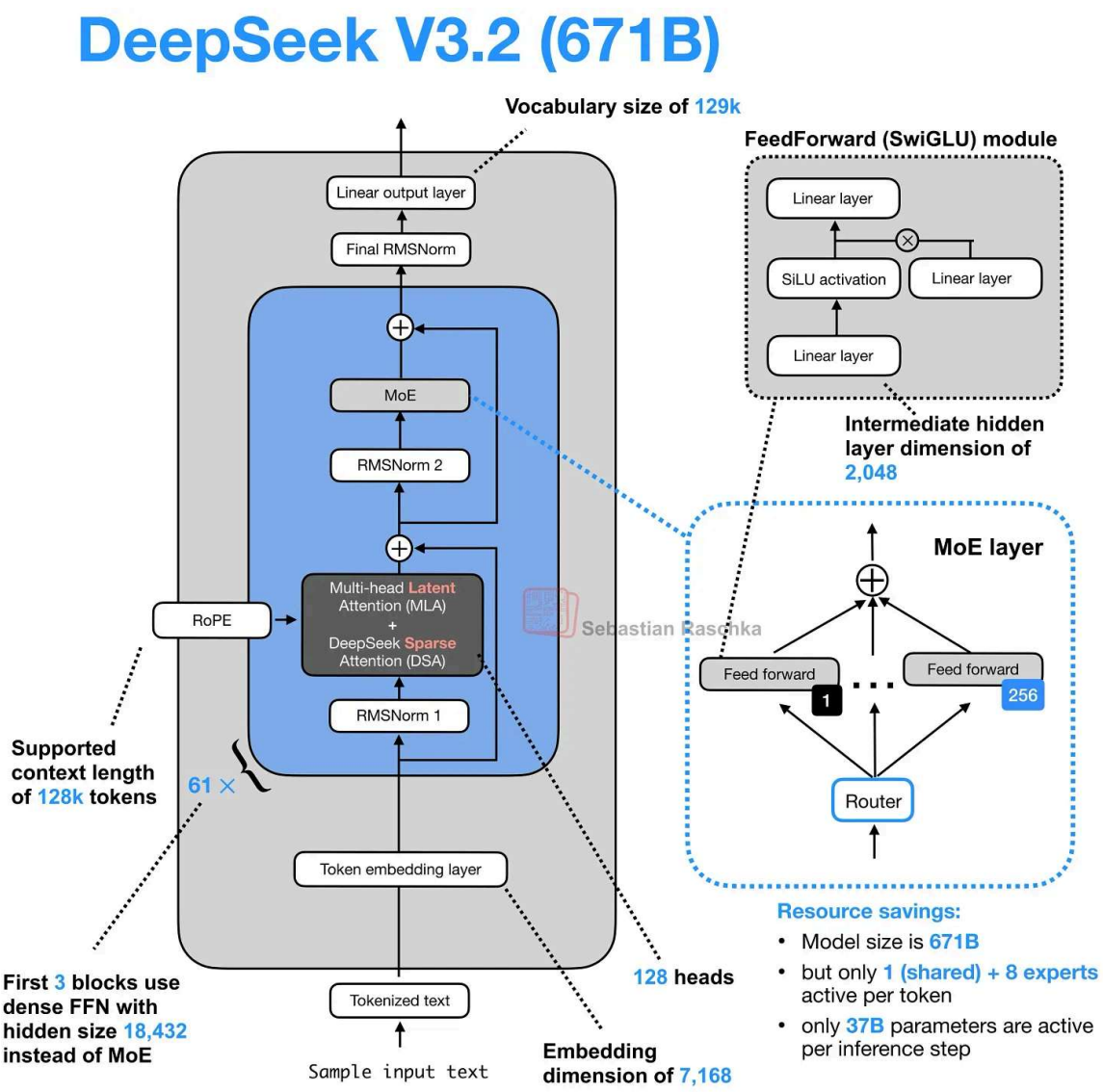


Figure 20: DeepSeek V3.2 is the obvious reference point, because this is the model family most closely associated with the sparse-attention idea.

The sparse pattern is not random. The first stage is a lightning indexer that scores prev tokens for each new query token. It uses MLA's compressed token representations anc computes a learned similarity score over the prior context, so the model can rank whicl earlier positions are worth revisiting.

The second stage is a token selector. It keeps only a smaller high-scoring subset, for example, a top-k set of past positions, and turns that subset into the sparse attention m So the main point is that DeepSeek Sparse Attention does not hard-code the sparsity pattern. It learns which past tokens to keep.

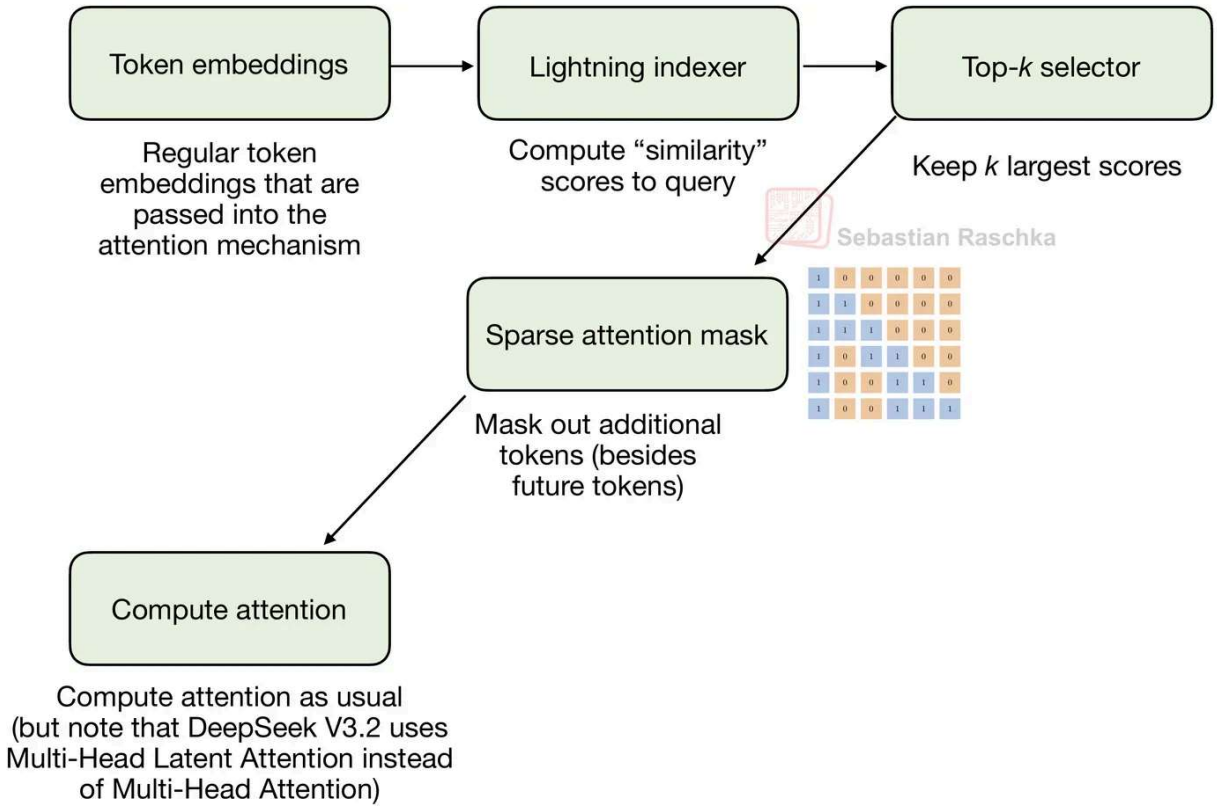


Figure 21: The mechanism consists of a lightning indexer that scores prior tokens and a selector that keeps only a smaller subset for attention (Original source: *From DeepSeek V3 to V3.2: Architecture, Sparse Attention, and RL Updates*).

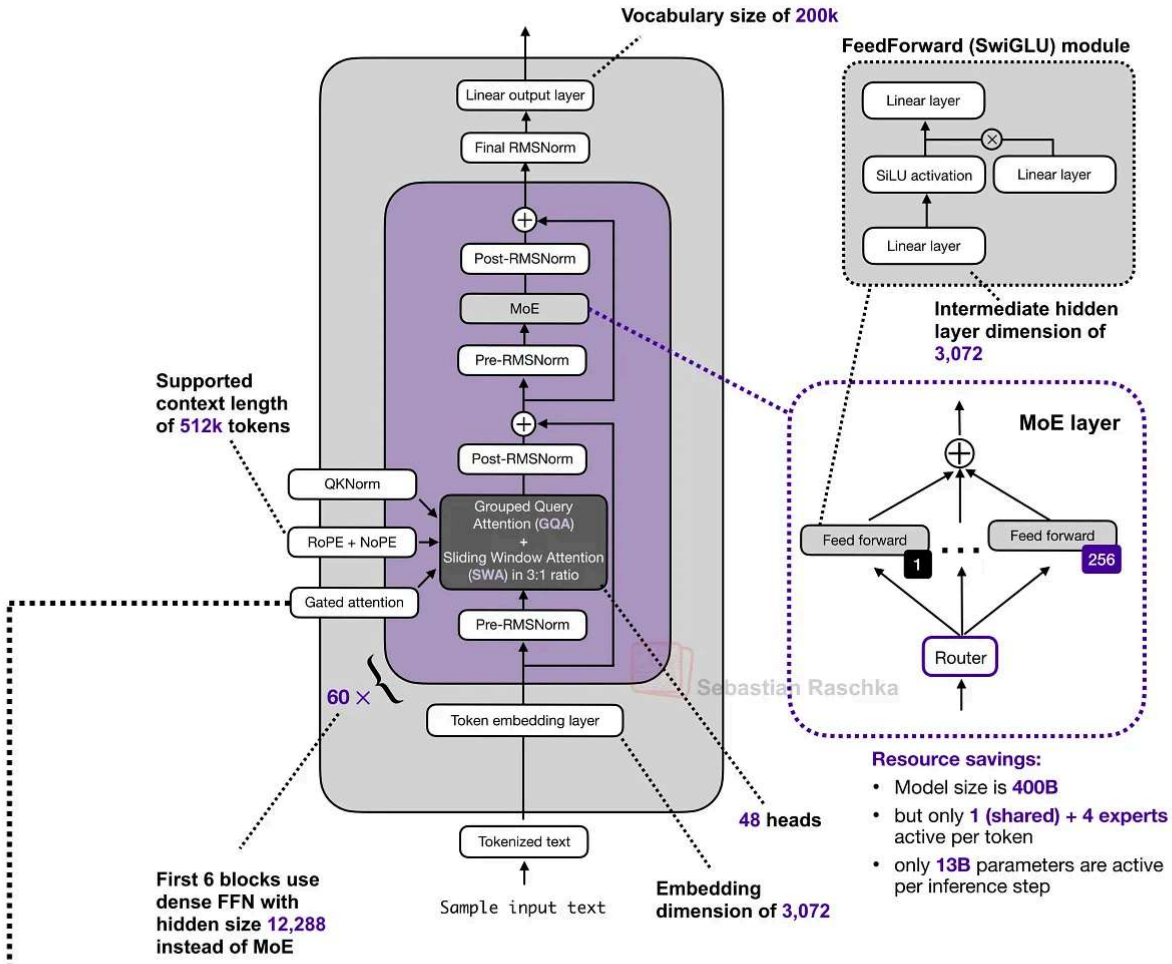
DeepSeek Sparse Attention is relatively new and relatively complicated to implement, v is why it has not been so widely adopted as Grouped-Query Attention (GQA) yet.

6. Gated Attention

Gated attention is best understood as a modified full-attention block rather than as a separate attention family.

It usually appears inside hybrid stacks that still keep an occasional full-attention layer f exact content retrieval, but add a few stability-oriented changes on top of an otherwise familiar scaled dot-product attention block.

Arcee AI Trinity Large (400B)




```python
def attn_trinity(x, Wq, Wk, Wv, Wo, Wg, *, h):
    B, T, D = x.shape
    dh = D // h
    q = (x @ Wq.T).view(B, T, h, dh).transpose(1, 2)
    k = (x @ Wk.T).view(B, T, h, dh).transpose(1, 2)
    v = (x @ Wv.T).view(B, T, h, dh).transpose(1, 2)

    q = rmsnorm(q)  # QK-Norm
    k = rmsnorm(k)

    o = sdpa(q, k, v) # standard scaled dot product attention

    ### Gating ###
    g = torch.sigmoid(x @ Wg.T).view(B, T, h, dh).transpose(1, 2)
    o = o * g  # Elementwise gate AFTER attention before Wo output projection
    ##############

    return (o.transpose(1, 2).reshape(B, T, D)) @ Wo.T
```

Figure 22: Trinity Large is a useful comparison because gated attention is not only a Qwen idea (more on that later). Here the gate appears after the scaled dot-product attention output and before the output projection in a different long-context architecture (Original source: *A Dream of Spring for Open-Weight LLMs*).

6.1 Where Gated Attention Appears

The Qwen3-Next and Qwen3.5 architectures show that recent hybrids (covered in the section) do not replace attention everywhere. Instead, they replace most attention laye a cheaper alternative and keep a smaller number of full-attention layers in the stack.

Those remaining full-attention layers are where gated attention typically appears. Qwe Next and Qwen3.5 use it together with Gated DeltaNet in a 3:1 pattern.

But hybrid architectures aside, Trinity uses a related gating idea in a more conventional attention stack, as shown in the previous figure above.

6.2 Gated Attention Relative To Standard Attenti

The gated attention block in Qwen-style hybrids or Trinity (not a hybrid) is essentially standard scaled-dot-product attention with a few changes on top. In the original Gated Attention paper, those changes are presented as a way to make the retained full-attent layers behave more predictably inside a hybrid stack.

The block still looks like standard (full) attention, but it adds:

1. an output gate that scales the attention result before it is added back to the residu
2. a zero-centered QK-Norm variant instead of standard RMSNorm for q and k,
3. partial RoPE.

These are not changes on the scale of MLA or linear attention but merely stability and changes applied to an otherwise familiar attention block.

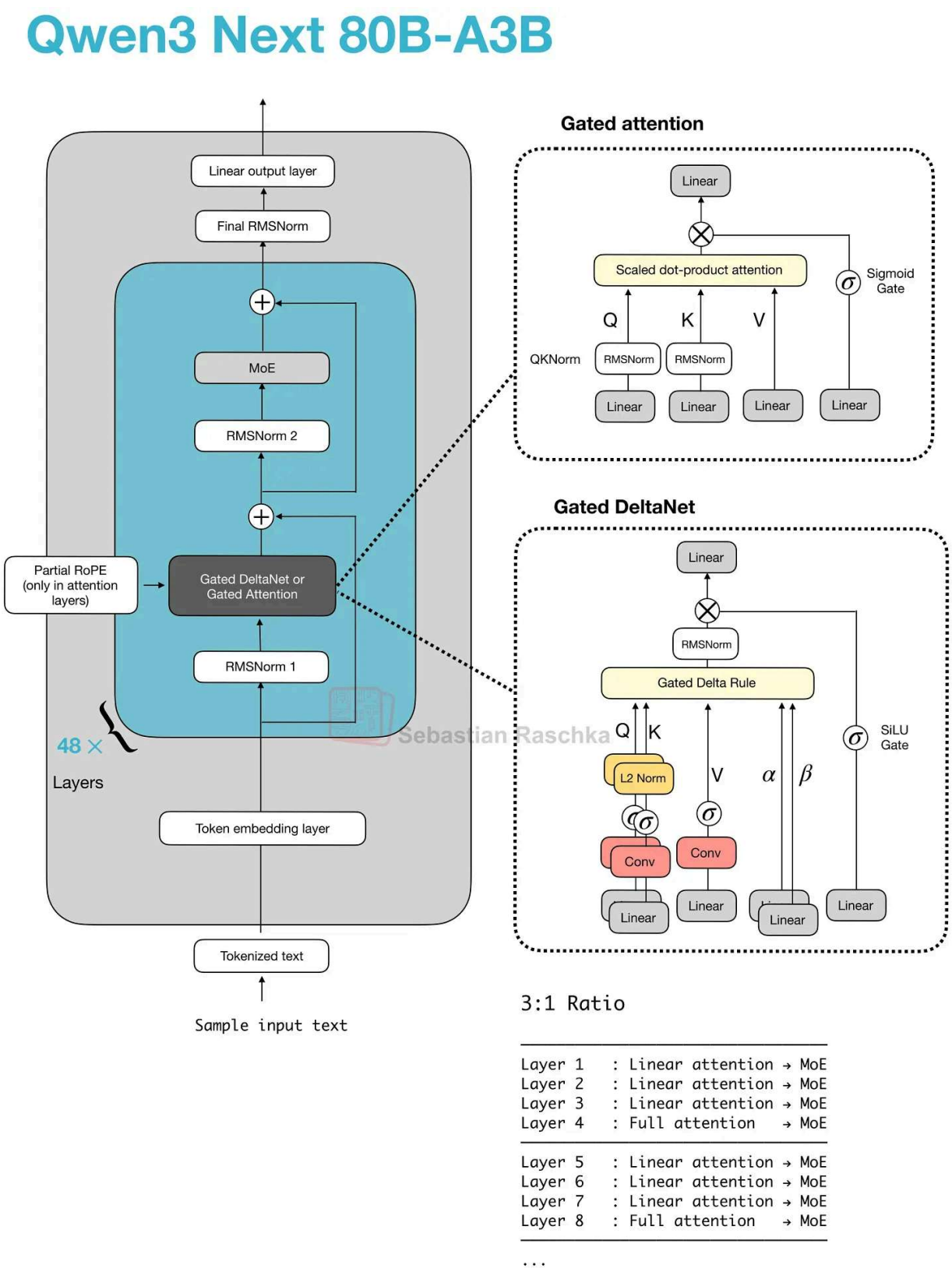


Figure 23: In Qwen3-Next and Qwen3.5, gated attention appears as the full-attention layer that periodically breaks up runs of Gated DeltaNet blocks.

Note that the figure above also includes Gated DeltaNet, which we will cover in the nex section below.

7. Hybrid Attention

Hybrid attention is a broader design pattern rather than a specific, single mechanism. T overall idea is to keep a transformer-like stack, but replace most of the expensive full- attention layers with cheaper linear or state-space sequence modules.

The motivation is long-context efficiency. Full attention grows quadratically with seque length, so once models move to contexts like 128k, 256k, or 1M tokens, attention mem and compute become expensive enough that using cheaper sequence modules in mo layers while keeping only a smaller number of heavier retrieval layers starts making mo sense. (Note that this comes with a bit of a modeling performance trade-off, though.)

In Qwen3-Next, this pattern appears as a 3:1 mix of Gated DeltaNet and Gated Attenti blocks. Gated DeltaNet is also closely related to Mamba-2 (see the *Gated Delta Netwo Improving Mamba2 with Delta Rule* paper, for instance), and the mechanism can be re a DeltaNet-style fast-weight update combined with Mamba-style gating. Later archite keep the same overall idea but swap in other lightweight sequence mixers, such as Kin Delta Attention, Lightning Attention, or standard Mamba-2.

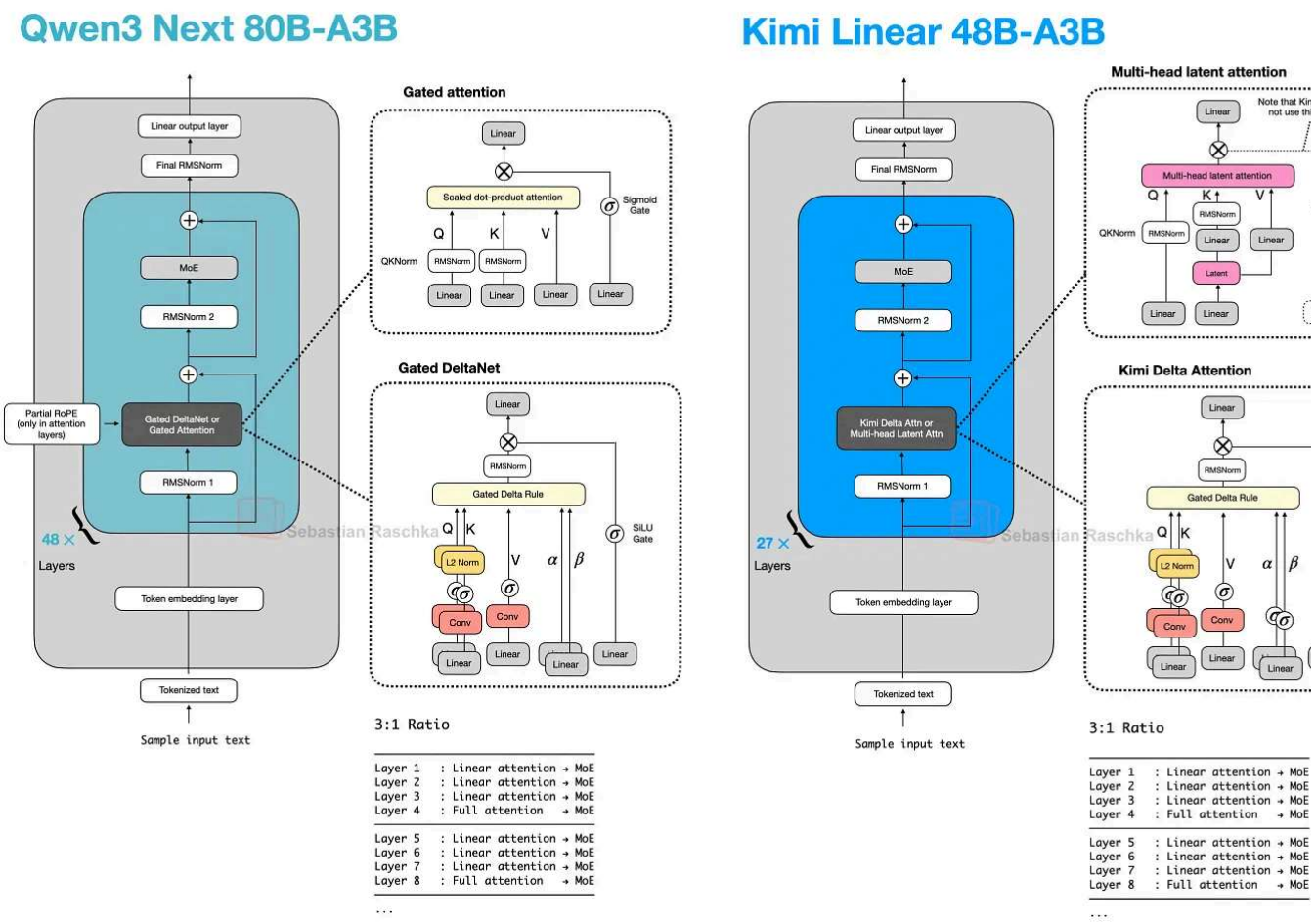


Figure 24: The basic hybrid pattern, where most blocks are cheaper sequence mixers and every fourth block restores a heavier attention layer (Original source *The Big LLM Architecture Comparison*).

7.1 Gated DeltaNet in Qwen3-Next

To my knowledge, the first prominent example of a close-to-flagship LLM with hybrid attention was Qwen3-Next in 2025, which does not remove attention completely but r three Gated DeltaNet blocks with one Gated Attention block.

Here, lightweight Gated DeltaNet blocks do most of the long-context work and keep m growth much flatter than full attention. The heavier gated-attention layer remains beca DeltaNet is less exact at content-based retrieval.

Inside a Gated DeltaNet block, the model computes query, key, and value vectors toget with two learned gates (α, β). Rather than forming the usual token-to-token attention m it writes to a small fast-weight memory using a delta-rule update. In rough terms, the m

stores a compressed running summary of past information, while the gates control hov much new information is added and how much previous state is retained.

That makes Gated DeltaNet a linear-attention or recurrent-style mechanism rather tha another tweak to MHA. Relative to Mamba-2, the close connection is that both belong linear-time gated sequence-model family, but Gated DeltaNet uses a DeltaNet-style fa weight memory update instead of the Mamba state-space update.

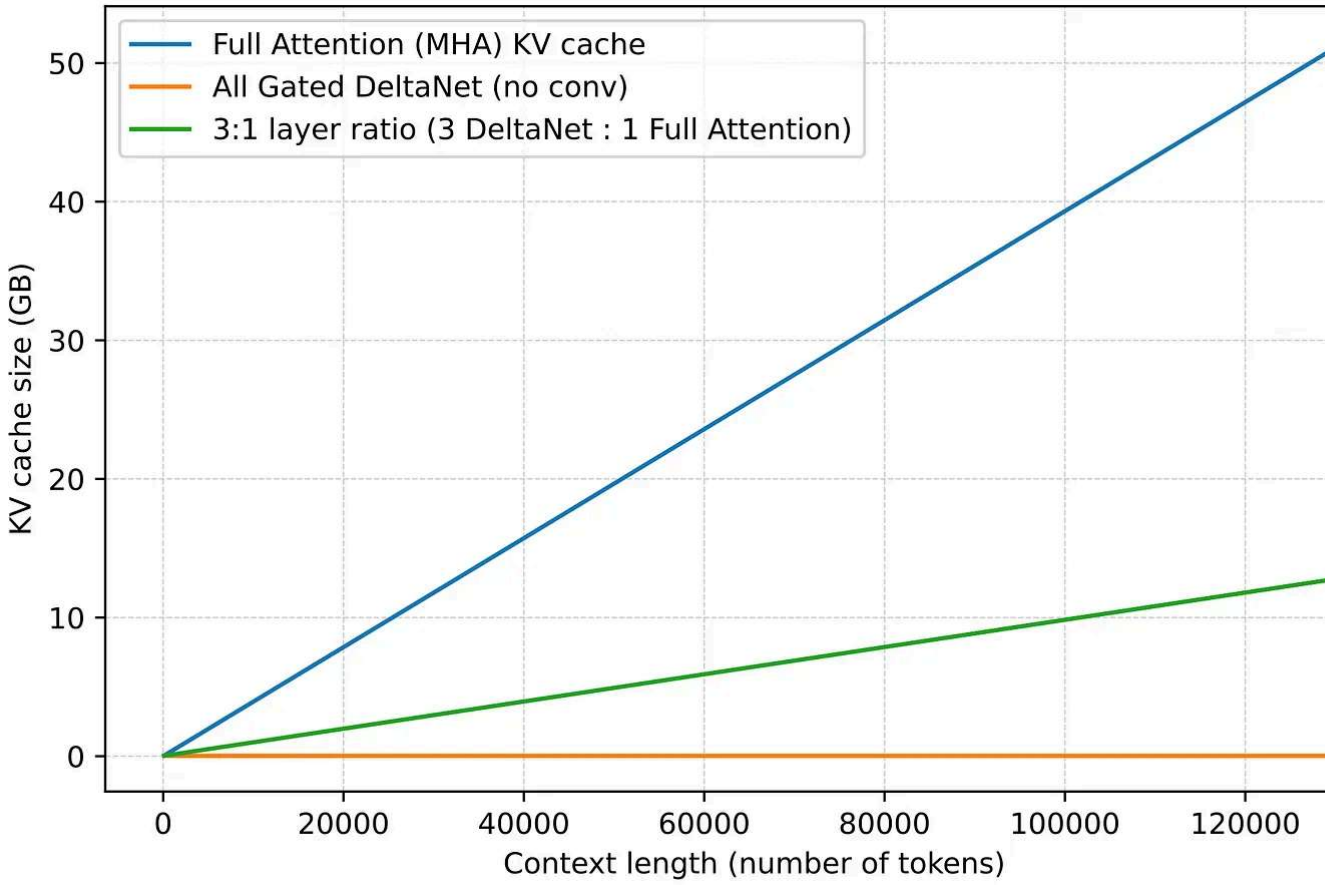


Figure 25: The practical motivation behind the hybrids is shown here in the memory curve. Hybrid stacks with Gated DeltaNet grow much more slowly with context length than ordinary full attention (Original source *LLMs-from-scratch* DeltaNet materials).

Qwen3.5 moves the former Qwen3-Next hybrid into Qwen's main flagship series, whic interesting move. This basically signals that the hybrid strategy is a success and that w see more models with this architecture in the future.

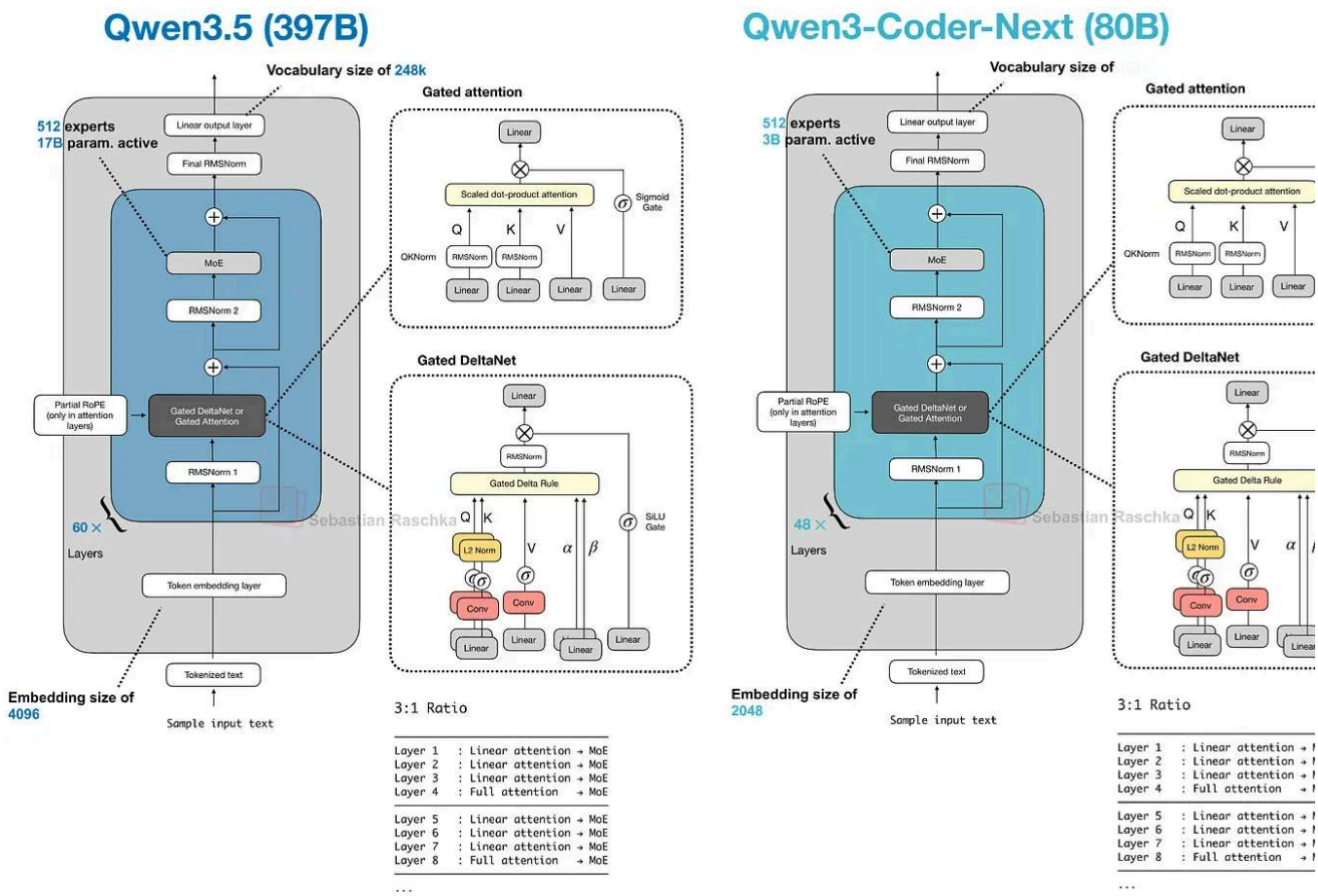


Figure 26: Qwen3.5 shows the Qwen team promoting the former Qwen3-Next side-branch into the main model line rather than leaving it as a one-off efficiency variant (Original source *A Dream of Spring for Open-Weight LLMs*).

7.2 Kimi Linear And Modified Delta Attention

Kimi Linear keeps the same broad transformer skeleton and the same 3:1 pattern, but i changes both halves of the recipe.

On the lightweight side, Kimi Delta Attention is a refinement of Gated DeltaNet. Where Qwen3-Next uses a scalar gate per head to control memory decay, Kimi uses channel- gating, which gives finer control over the memory update. On the heavier side, Kimi rep Qwen3-Next's gated-attention layers with gated MLA layers.

So, it's still the same broader pattern as in Qwen3-Next and Qwen3.5, but both ingredi (slightly) change. I.e., most layers are still handled by a cheaper linear-style mechanism periodic heavier layers still remain for stronger retrieval.

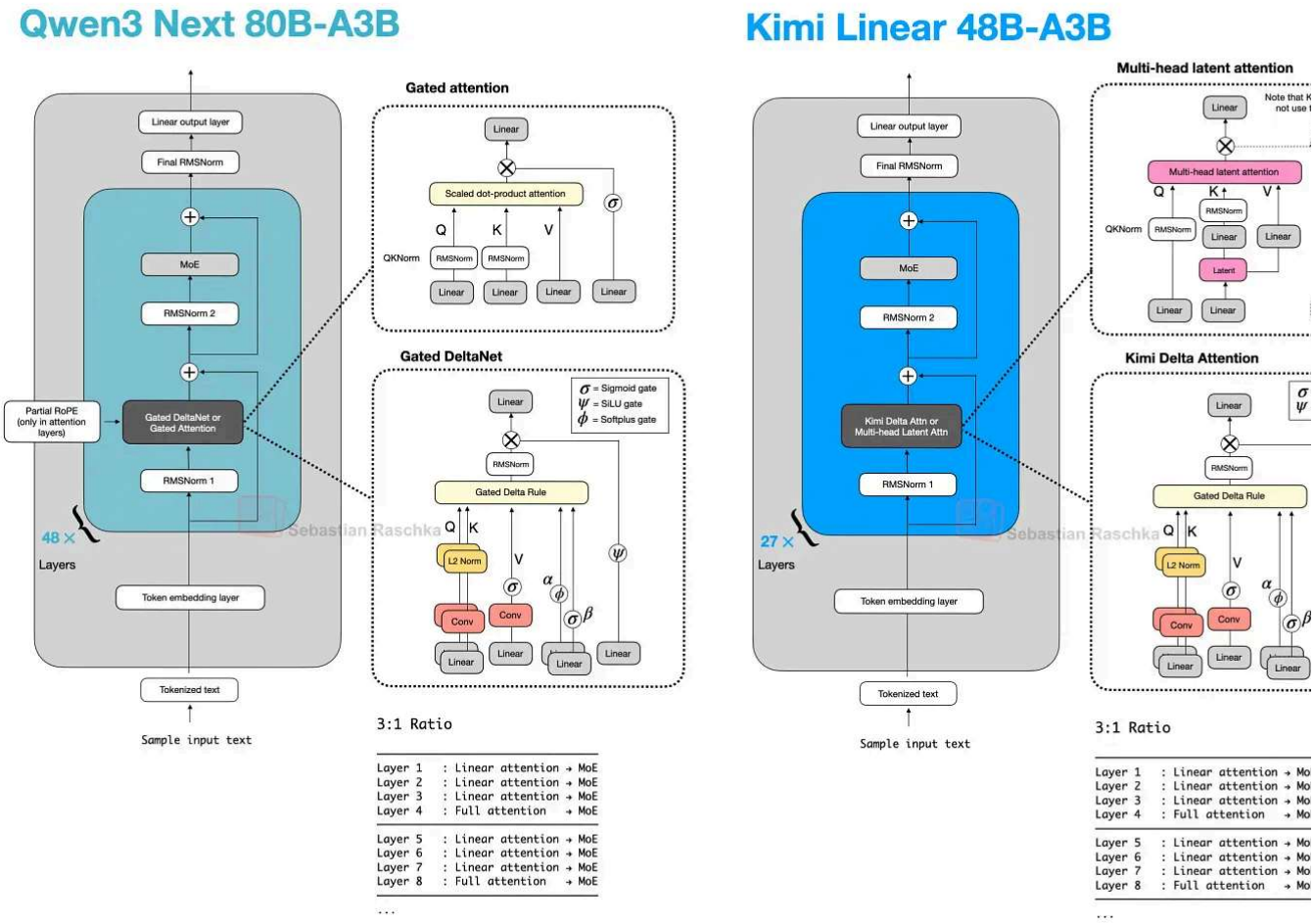


Figure 27: Kimi Linear keeps the same overall hybrid pattern while changing both the lightweight side and the heavier attention side of the stack (Original source *The Big LLM Architecture Comparison*).

7.3 Ling 2.5 And Lightning Attention

Ling 2.5 shows another swap on the lightweight side. Instead of Gated DeltaNet, Ling u slightly simpler recurrent linear attention variant called Lightning Attention. On the hea side, it keeps MLA from DeepSeek.

Most sequence mixing happens in the cheaper linear-attention blocks, while a smaller number of heavier layers remain to preserve stronger retrieval. The difference is that th specific lightweight mechanism is now Lightning Attention rather than DeltaNet or Kim Attention.

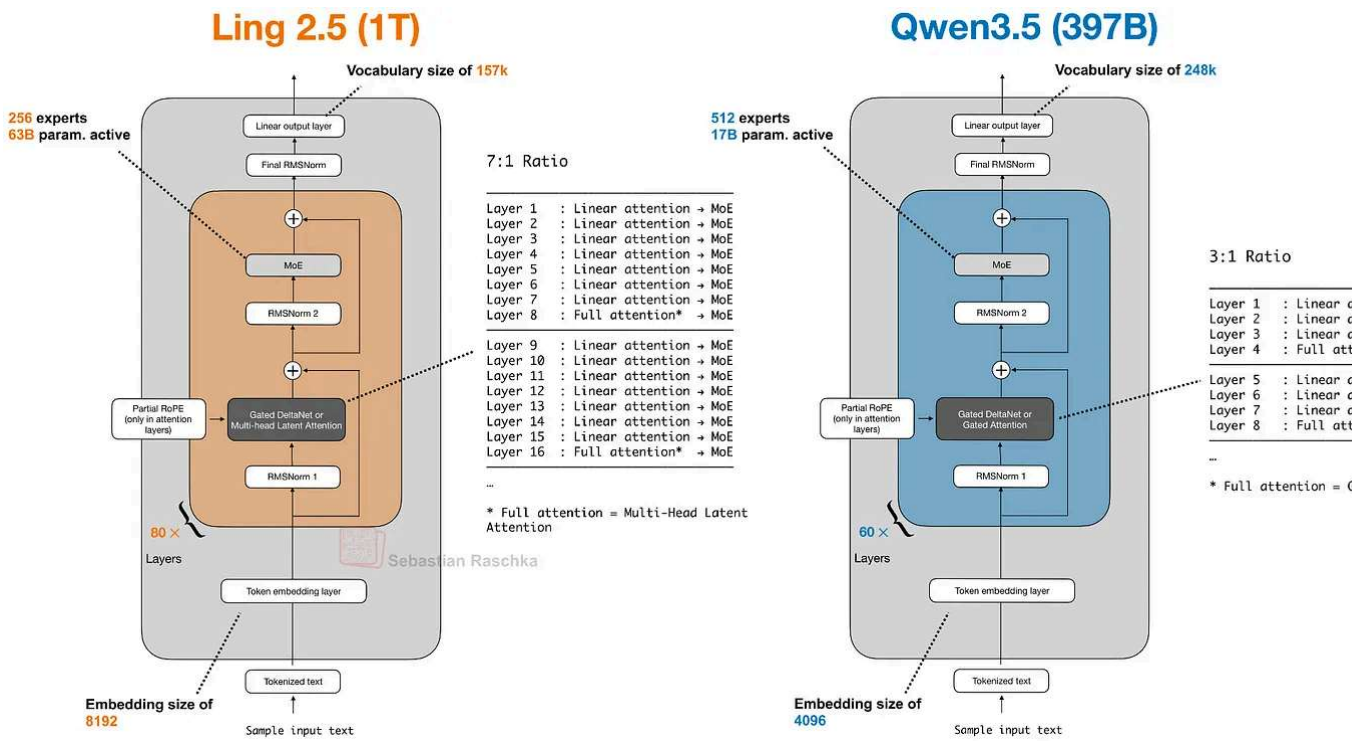


Figure 28: Ling 2.5 and Qwen3.5 are both linear-attention hybrids, even though Ling swaps in Lightning Attention and MLA instead of the Qwen recipe (Original source *A Dream of Spring for Open-Weight LLMs*).

Ling 2.5 is aimed more at long-context efficiency than at absolute benchmark leadersl According to the Ling team, it was reported as substantially faster than Kimi K2 at 32k tokens, which is the practical payoff these hybrids are aiming for.

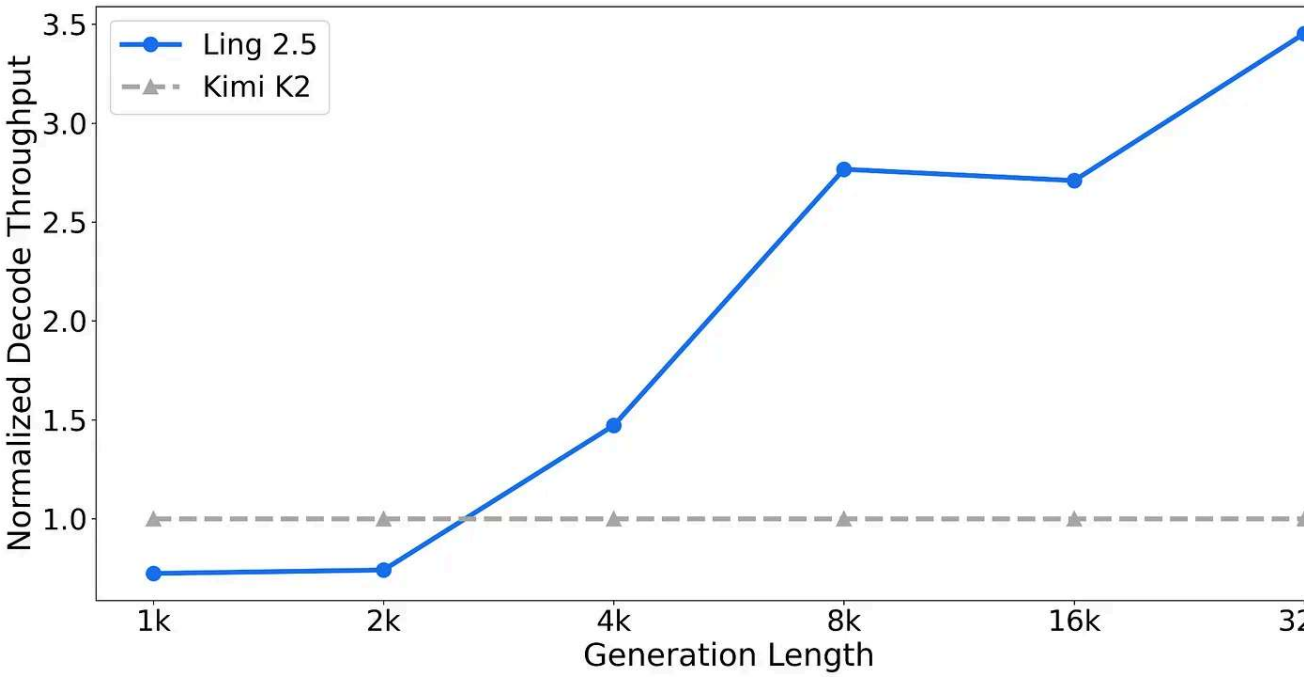


Figure 29: Ling 2.5 was presented as a strong efficiency upgrade, with much higher 32k-token throughput than Kimi K2 at the same 1-trillion-parameter scale (Original source *Ling 2.5 model hub page*).

Nemotron And Mamba-2

Nemotron pushes the pattern further away from the transformer baseline. Nemotron 3 is a Mamba-Transformer hybrid that interleaves Mamba-2 sequence-modeling blocks sparse MoE layers and uses self-attention only in a small subset of layers.

This is a more extreme version of the same basic tradeoff discussed above. Here, the lightweight sequence module is a Mamba-2 state-space block rather than a DeltaNet-fast-weight update, but the basic tradeoff is similar.

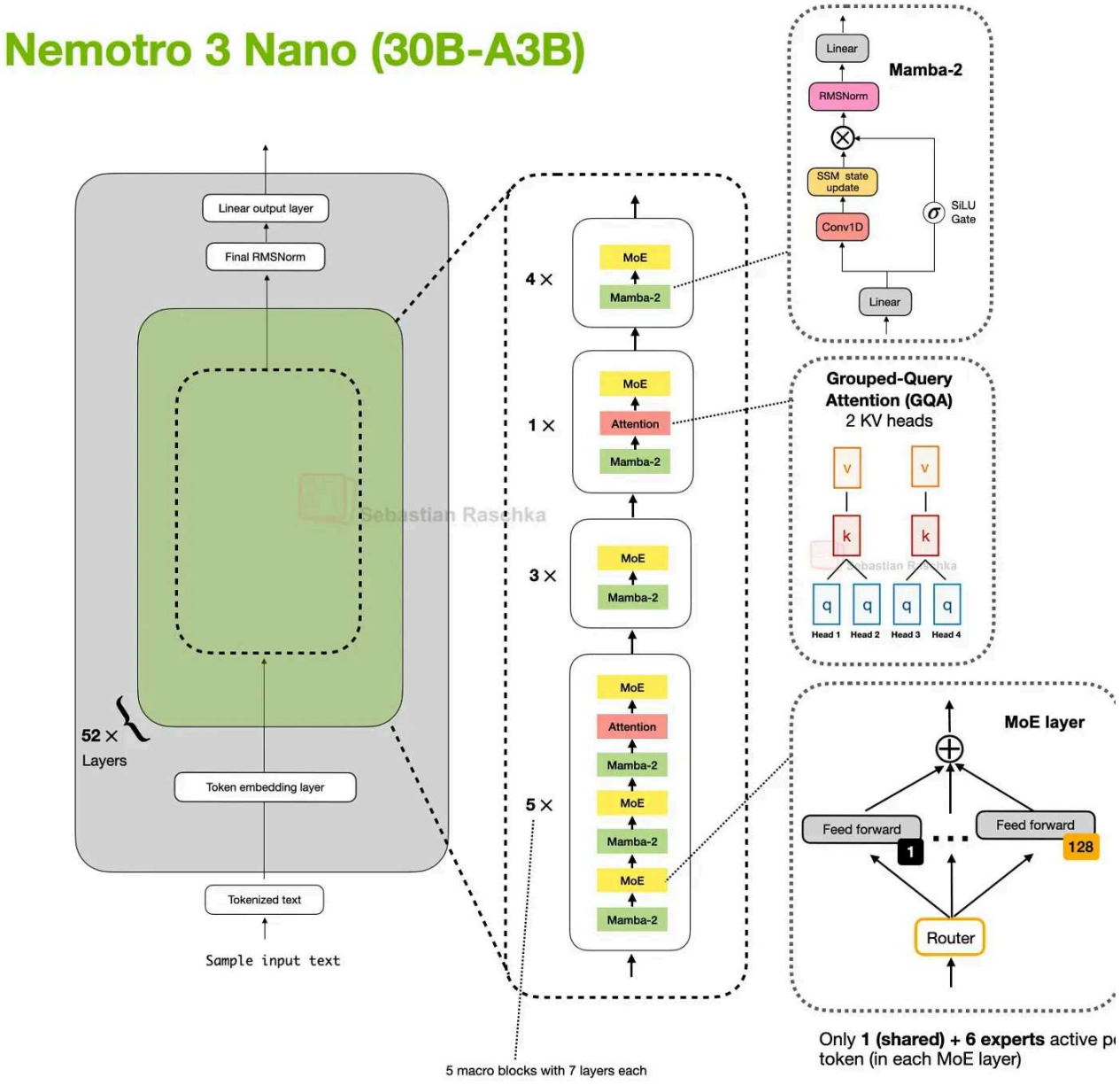


Figure 30: Nemotron 3 Nano uses Mamba-2 for most of the sequence modeling work, with self-attention only appearing in a small subset of layers (Original source *The Big LLM Architecture Comparison*).

The larger Nemotron 3 Super keeps the Mamba-2 hybrid attention approach and adds efficiency-oriented changes such as latent MoE and shared-weight multi-token predic (MTP) for speculative decoding.

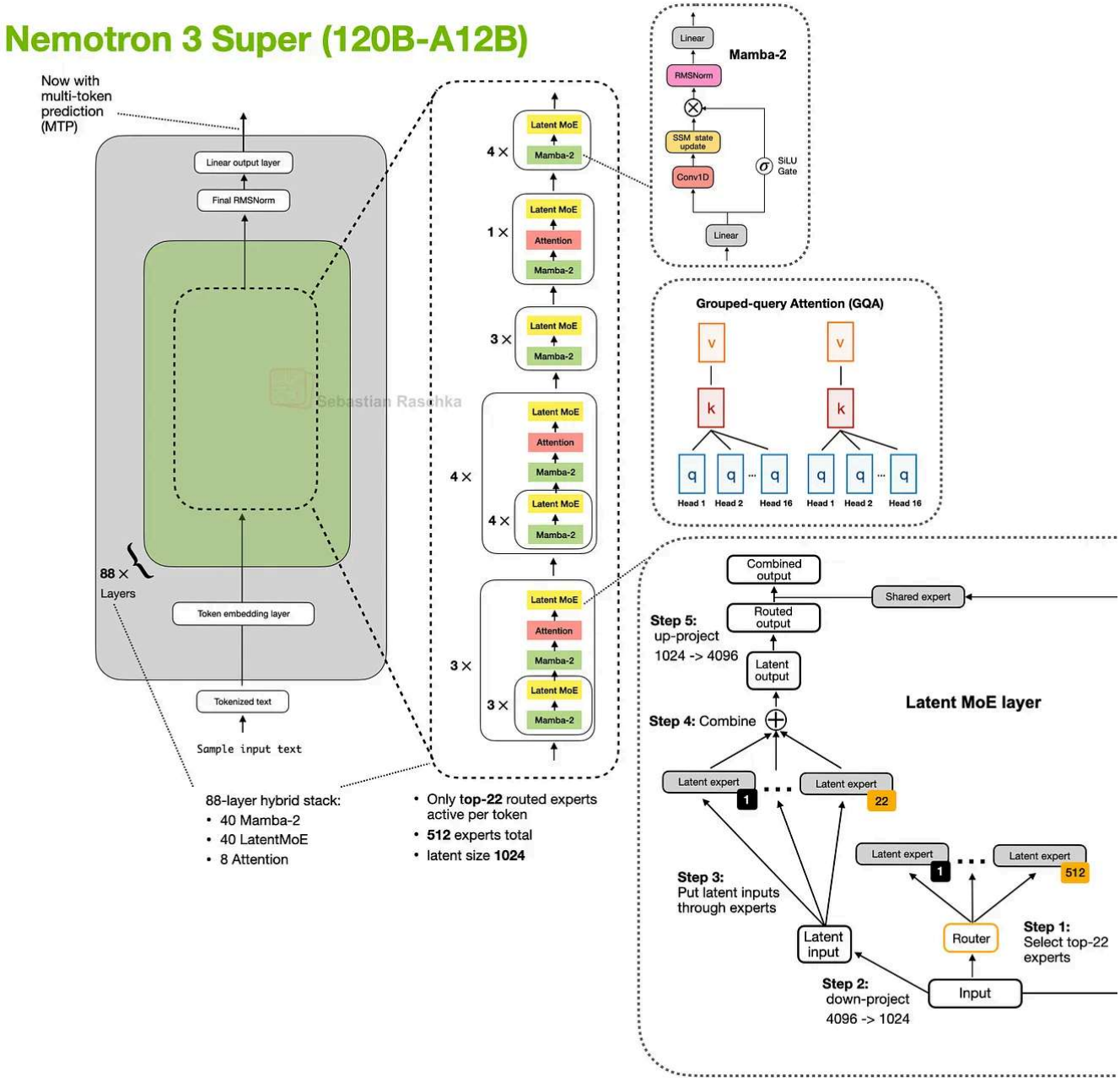


Figure 31: Nemotron 3 Super keeps the Mamba-2 hybrid attention pattern while adding latent MoE and shared-weight MTP on top (Original source *The Big LLM Architecture Comparison*).

Conclusion

Of course, there are many more (mostly niche) attention variants throughout the literat that I haven't covered here. The focus of this article was on those that are currently use state-of-the-art (open-weight) models.

In particular, I am looking forward to (1) seeing the brand new Mamba-3 layers getting integrated into the aforementioned hybrid architectures (replacing Gated DeltaNet) an attention residuals being used in general.

In practice, you may also wonder what the "best" architecture is at the moment. This is to answer, as there are no public experiments that train different architectures on the sa training data etc.

Hence, we can currently only answer what the best (trained) model choice is for a give problem. In my opinion, hybrid architectures are still a novelty, and the main selling poin mainly (long-context) efficiency versus just modeling performance. Hence, I think they great candidate for agent contexts (like OpenClaw).

Personally, I think the problem with hybrid architectures is also that the inference stack not quite as optimized, yet, and I find that I get better tok/sec throughput when running locally using more classic setups like GPT-OSS with grouped-query attention.

Anyways, I am curious to see what DeepSeek V4 has in store, since DeepSeek has bee quite the reliable trend-setter in the recent 2 years.

Discussion about this post

I got quite confused for the classification of these attention mechanisms. I basically th hybrid Attention are those MLA+Sparse Attention or SWA+GQA with different ratios (Attention plus Delta Gate Attention or so....but your post states that Hybrid Attention Transformer block replacement with linear Attention or State Space Module. It is a kin Hybrid, but is more like a new design of the new LM Architecture. BTW, I feel like Kimi Attention Residuals can be part of the future Hybrid list, if AttnRes can be proved to b compatible with GQA or MLA then.

REPLY

1 reply by Sebastian Raschka, PhD

prcanegaly Mar 22

Liked by Sebastian Raschka, PhD

Very dense content. Going thorough this slowly, to understand.

REPLY

13 more comments...